

MATH5320 - HMWK 11

Assignment 7 Q2 & Ass 9 Q4

2. Historical VaR, relative changes

Continuing with the settings in the previous homeworks, namely A being AAPL, C being CAT, and P being the portfolio consisting of 24679 shares of AAPL, and 171 shares of CAT.

Compute the 99% 5 day historical VaR and the 97.5% 5 day historical ES for a \$10,000 investment in A , C , and P on each day in the last 20 years. Do this for each date d by applying the previous 5 years of daily log returns to the position on that date (windowed).

How do the historical VaR and ES compare to the previous results?

Would it matter if the VaR for P was computed from the historical time series changes for P or from the historical time series changes for A and C applied to the underlying stocks in the portfolio? Why or why not?

4. Short portfolio, VaR and some ES

Setting as in previous VaR calculation problems.

Repeat the VaR calculations for short portfolios. Instead of having positive positions in A , C , and P , consider having equal but negative positions in each. Use the 5 year period parameter estimates.

Work with shorts as follows. If you have \$1 and buy a stock that costs \$1, you then have \$1 worth of stock. If you have \$1 and short the stock, you then have \$2 in cash and -\$1 worth of stock. We ignore interest rate impact, so the cash component doesn't change in value when computing VaRs.

Compute VaR for \$10,000 invested in each on each date. Compute using formulas (assuming portfolios follow GBM).

How do the VaRs for short portfolios compare to those for long portfolios? Which are more risky, and why?

How do the VaRs compare for a short position in P when computing using the volatility estimates from the history of P vs pricing the portfolio by modeling A and C ? To answer this, compare the portfolio VaR computed above with the Monte Carlo VaR computed by directly modeling A and C .

5. Short portfolio, formula ES

Work out a formula for the ES of a short position in a portfolio following GBM. Use it to compare the 97.5% ES on a short position to the 99% VaRs computed using formulas in the previous problem (i.e. - for A , C and P across history calibrated to the last 5 years of data, equally weighted).

Are the 97.5% ESs and 99% VaRs on short portfolios close to each other?

1. Backtesting, 5 year window

Back test the GBM based VaR estimates for long and short portfolios by counting the number of times the VaR on each date is exceeded by the subsequent 5 day change in each 1 year window. Use the VaRs based on 5 year period parameter estimates. Do this for long and short positions in A , C , and P , as was done in homework 9.

What is the expected number of exceptions and how well do the number of exceptions match the expected number of exceptions?

The fixed-share portfolio is:

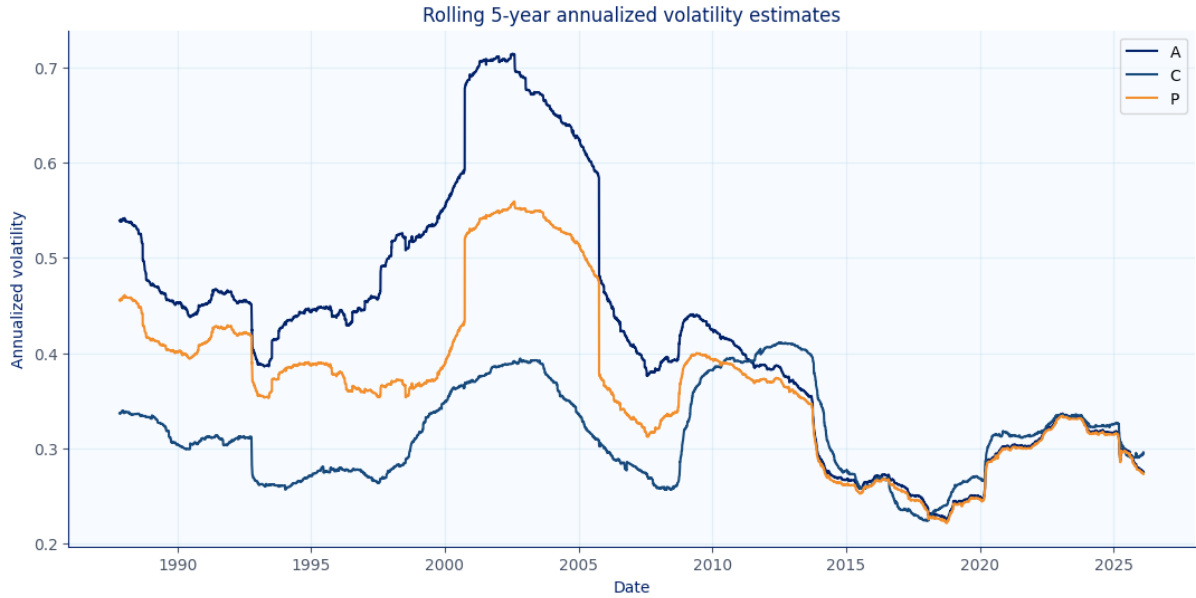
$$P_t = 24679A_t + 171C_t,$$

As shown in the set-up from Week 7 that was used AGAIN on week 9.

For each of A , C , and P , we compute daily log returns:

$$a_t = \log \left(\frac{S_t}{S_{t-1}} \right).$$

For each backtest date t , the previous 5 years of daily log returns are used to estimate the GBM mean and variance. With 252 trading days per year, the rolling estimation window is 1260 daily observations.



The fitted 5-day GBM return distribution is, which obtain literally by scaling:

$$R_{t,t+5} \sim N(5\hat{\mu}_t, 5\hat{\sigma}_t^2).$$

Then this means that for a long \$10,000 position, realised 5-day loss is as follows:

$$L_{t,t+5}^{\text{long}} = 10000(1 - e^{R_{t,t+5}}).$$

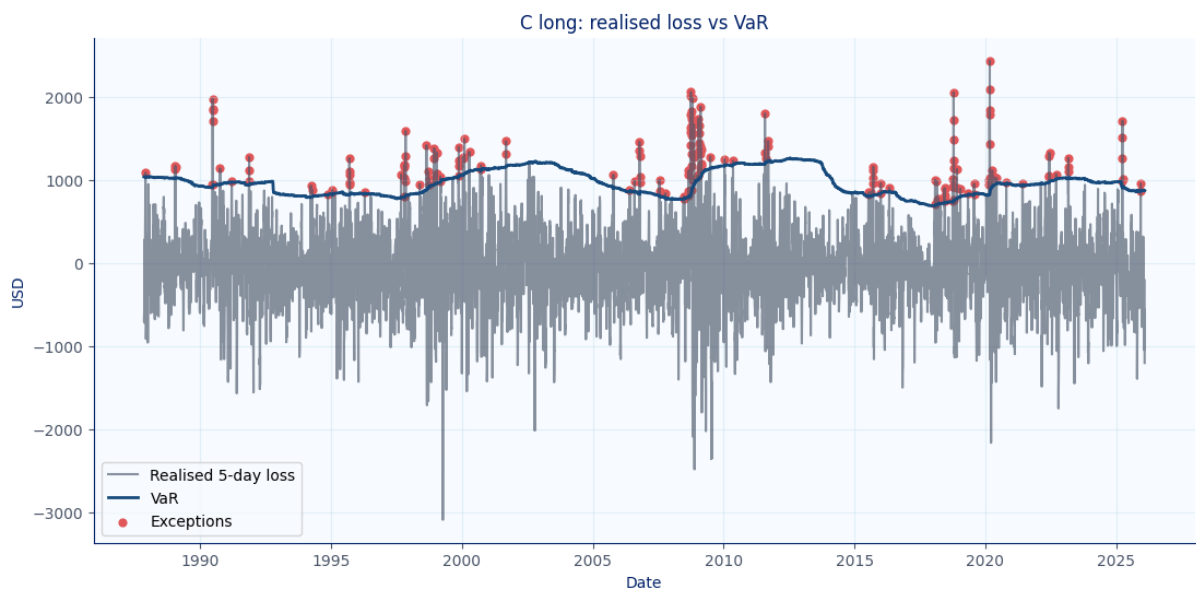
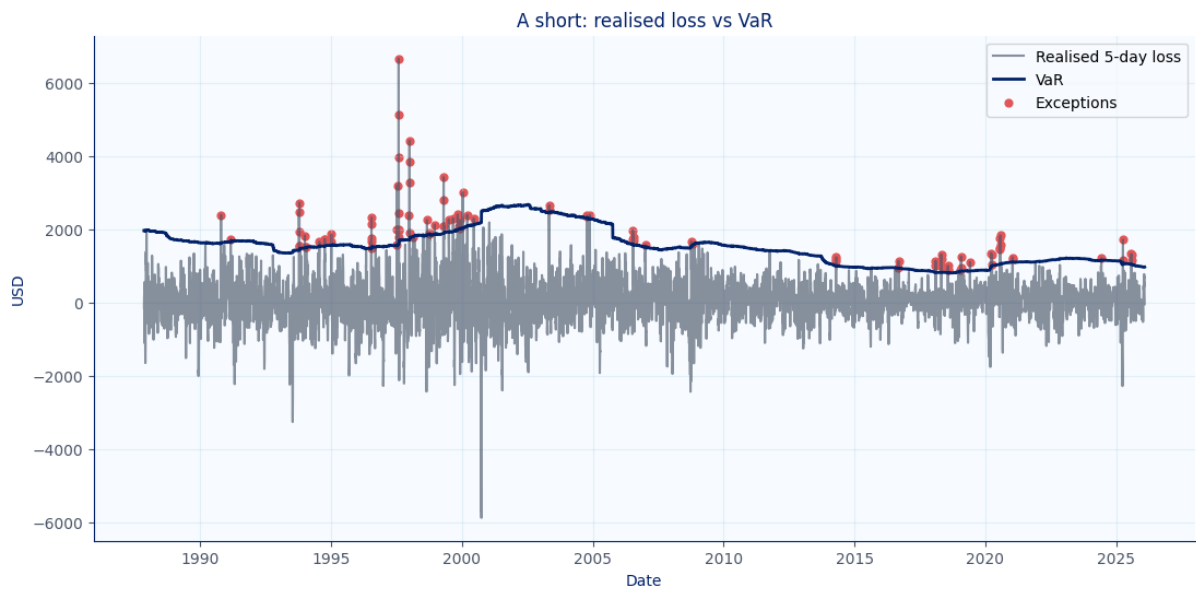
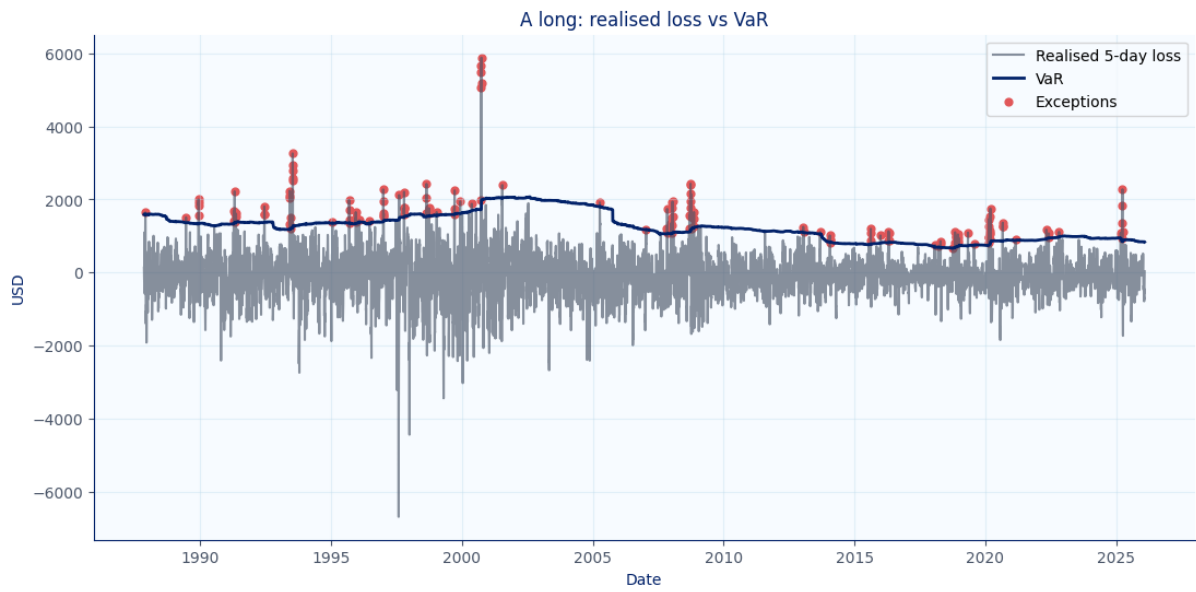
For a short \$10,000 position, realised 5-day loss is, and this is the opposite as we gain from a short position when the underlying goes down:

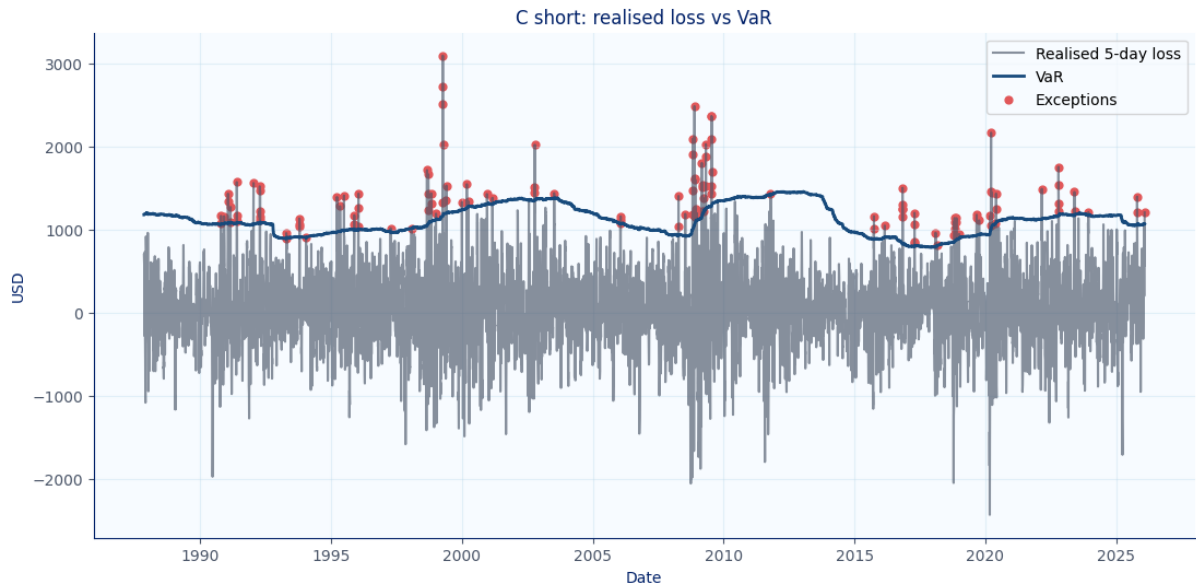
$$L_{t,t+5}^{\text{short}} = 10000(e^{R_{t,t+5}} - 1).$$

An exception occurs when the realised subsequent 5-day loss exceeds the VaR estimated on date t :

$$L_{t,t+5} > \text{VaR}_t.$$

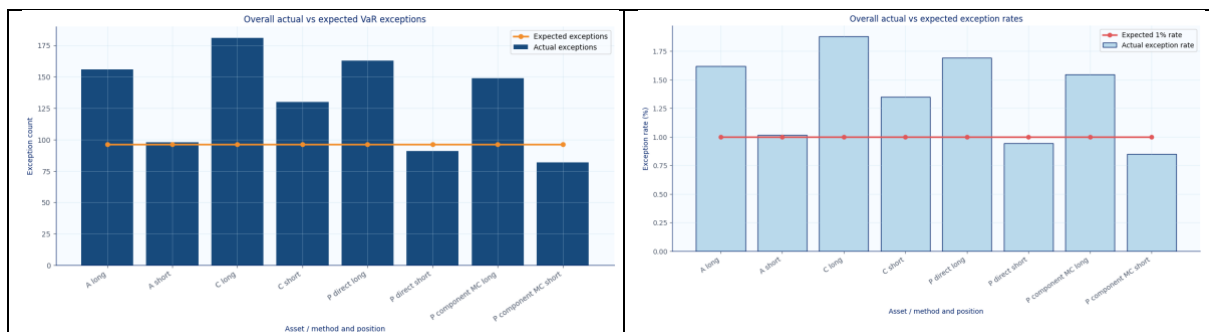
Now, we refer to the below:





Overall, we note that there are 9,633 backtest dates, and because there are 99% VaR backtesting, the exception rate should be 1%, so therefore, the expected number should be 96.33. We take note of the 5y rolling window results as below:

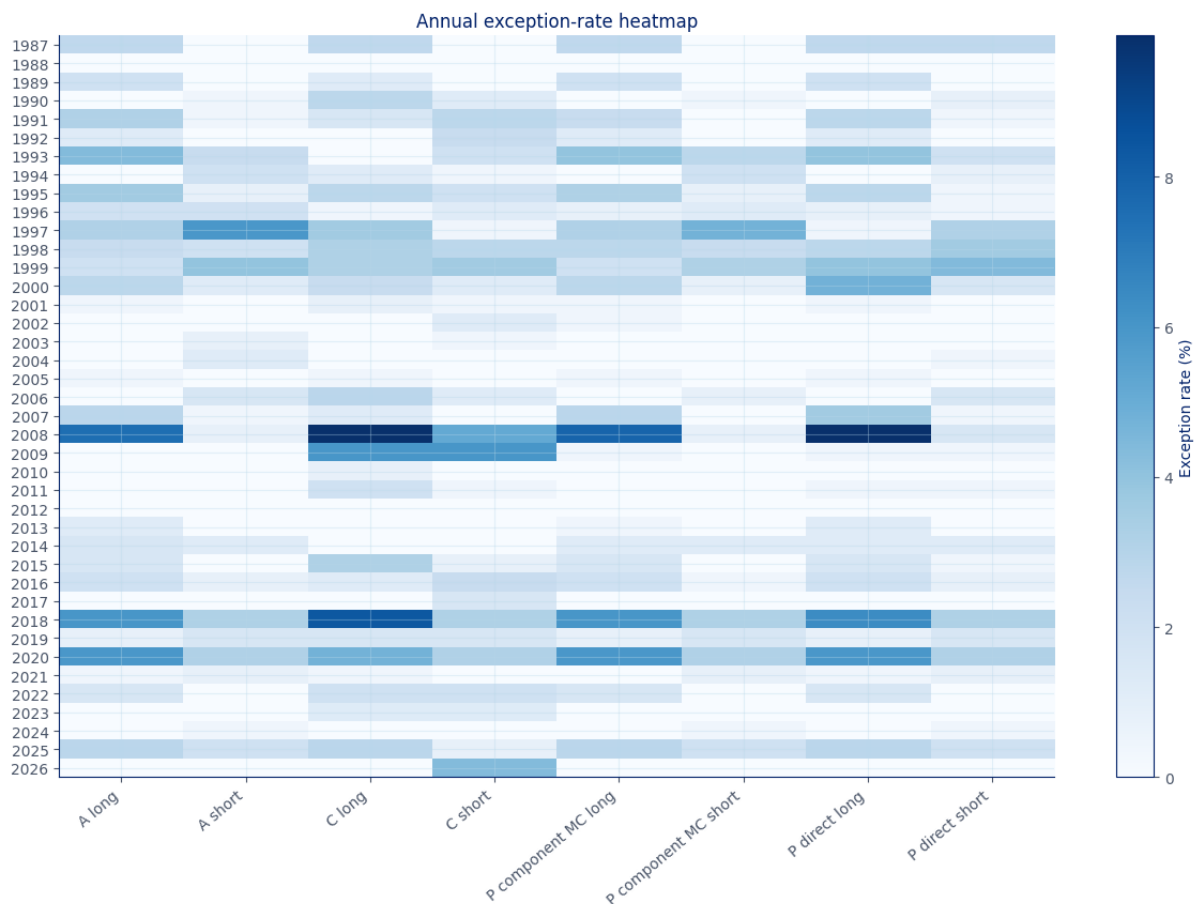
	Estimator	Asset / method	Position	Backtest dates	Expected exceptions	Actual exceptions	Exception rate	Expected exception rate
0	5Y rolling	A	long	9633	96.330000	156	0.016194	0.010000
1	5Y rolling	A	short	9633	96.330000	98	0.010173	0.010000
2	5Y rolling	C	long	9633	96.330000	181	0.018790	0.010000
3	5Y rolling	C	short	9633	96.330000	130	0.013495	0.010000
4	5Y rolling	P direct	long	9633	96.330000	163	0.016921	0.010000
5	5Y rolling	P direct	short	9633	96.330000	91	0.009447	0.010000
6	5Y rolling	P component MC	long	9633	96.330000	149	0.015468	0.010000
7	5Y rolling	P component MC	short	9633	96.330000	82	0.008512	0.010000



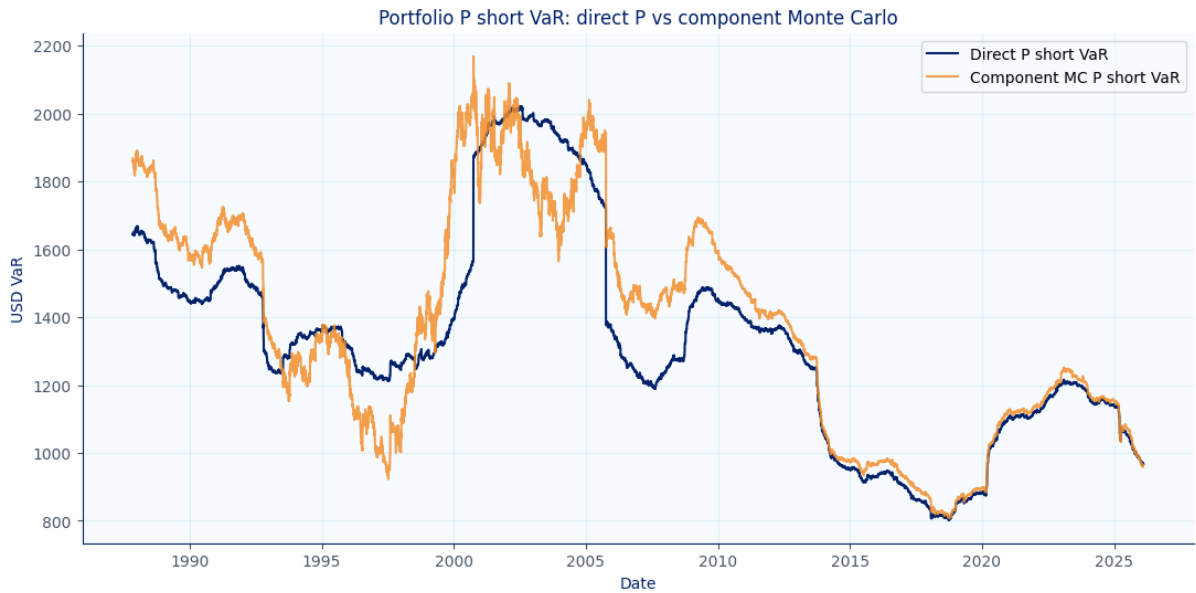
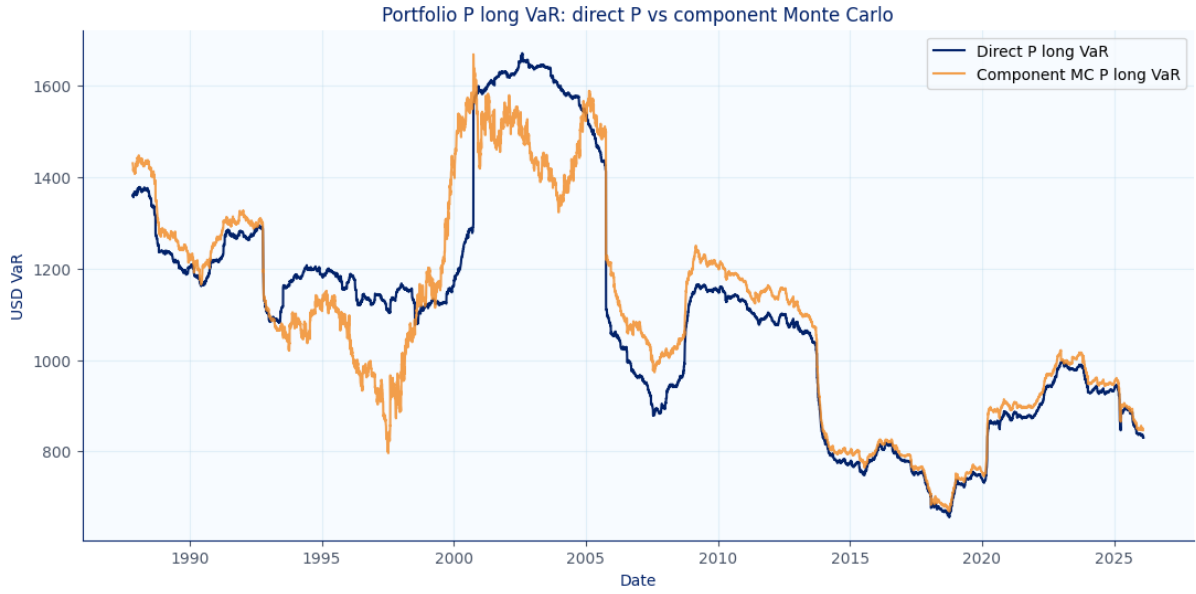
The expected number of exceptions is 96.33. The long-position results are above this expected number for every asset and portfolio method. AAPL long has 156 exceptions, CAT long has 181 exceptions, direct P long has 163 exceptions, and component Monte Carlo P long has 149 exceptions. These results cumulatively show that the GBM VaR model understates the downside tail risks for long positions. But, for short positions, the picture is more nuanced.

The short-position results are closer to the expected number. AAPL short has 98 exceptions, which is almost exactly equal to the expected value of 96.33. Direct P short has 91 exceptions and component Monte Carlo P short has 82 exceptions, both slightly below expectation. CAT short has 130 exceptions, which is above expectation, so the model still underestimates upside-tail risk for CAT. Overall, however, the short-position backtests are closer to the target exception rate than the long-position backtests.

Now, referring the below heat-map:



What this tells us is that the exceptions are not spread evenly through time, and that most of it is congregated or clustered in certain eras, like during the GFC. On an annual basis for instance, we should expect to see 2.52 days, but during the GFC we say 8+ for many portfolios. So a year with materially more than two or three exceptions indicates local VaR underestimation. The exception clustering suggests that the GBM model struggles during stressed or high-volatility periods. This is consistent with volatility clustering and fat-tailed equity returns, neither of which is fully captured by a simple rolling-window GBM model.



The above shows the direct vs Component MC methods. The direct method constructs the historical portfolio value:

$$P_t = 24679A_t + 171C_t,$$

then treats P_t as a single asset and fits a GBM to the log returns of P .

The component Monte Carlo method estimates the joint distribution of AAPL and CAT returns, including their covariance, then reprices the fixed-share portfolio under simulated AAPL and CAT shocks:

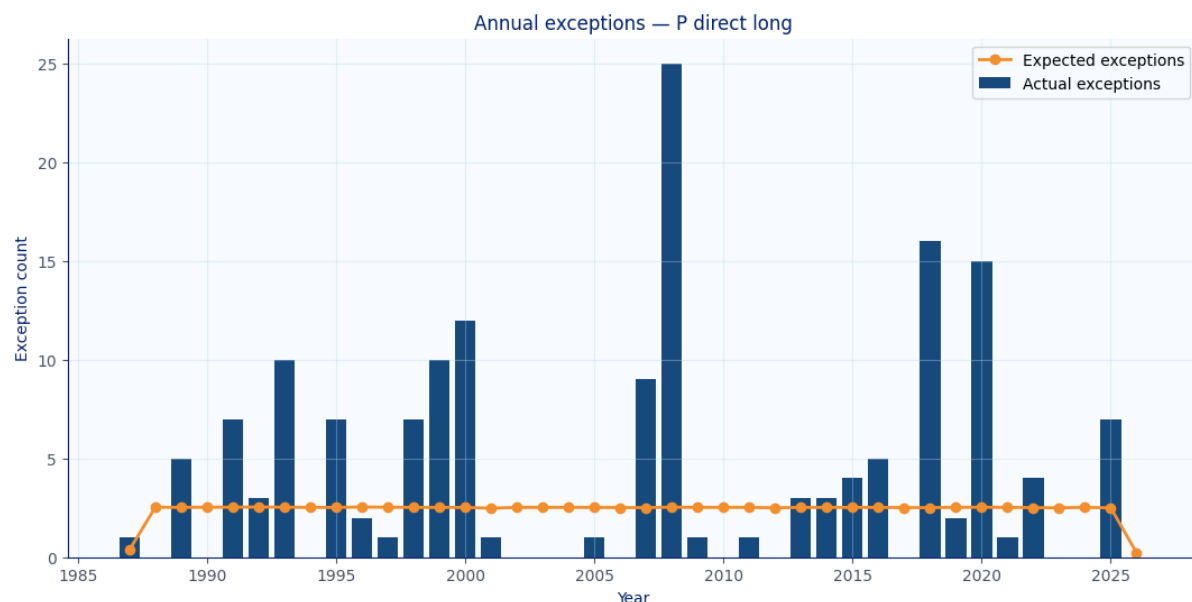
$$P_{t+5}^{(j)} = 24679A_te^{R_A^{(j)}} + 171C_te^{R_C^{(j)}}.$$

The component method is conceptually cleaner because P is a fixed-share portfolio. A weighted sum of two lognormal assets is not generally lognormal, so modelling P as one GBM is an approximation. The results are as follows:

4	P direct	long	9633	96.330000	163	0.016921	0.010000
5	P direct	short	9633	96.330000	91	0.009447	0.010000
6	P component MC	long	9633	96.330000	149	0.015468	0.010000
7	P component MC	short	9633	96.330000	82	0.008512	0.010000

We can see that for long P , both methods produce too many exceptions, meaning both underestimate downside risk. For short P , both methods produce slightly fewer exceptions than expected, meaning the short-portfolio VaR is slightly conservative. Notably, most of the clusters are during periods of stress like during the GFC and COVID.

For brevity, consider the P direct:



Overall, on an aggregated basis, the corrected AAPL/CAT portfolio backtest shows that GBM VaR performs better for short positions than for long positions in this sample. The model's largest weakness is the underestimation of downside losses for long positions, especially CAT and the portfolio. The annual exception heatmap shows that this failure is not uniform through time, exceptions cluster in stressed periods, which is consistent with volatility clustering and regime-dependent tail risk.

A similar story can be told for the 97.5% VaR.

97.5% VaR exception summary									
	VaR level	Estimator	Asset / method	Position	Backtest dates	Expected exceptions	Actual exceptions	Exception rate	Expected exception rate
0	0.975000	5Y rolling	A	long	9633	240.825000	238	0.024707	0.025000
1	0.975000	5Y rolling	A	short	9633	240.825000	180	0.018686	0.025000
2	0.975000	5Y rolling	C	long	9633	240.825000	306	0.031766	0.025000
3	0.975000	5Y rolling	C	short	9633	240.825000	273	0.028340	0.025000
4	0.975000	5Y rolling	P direct	long	9633	240.825000	250	0.025952	0.025000
5	0.975000	5Y rolling	P direct	short	9633	240.825000	181	0.018790	0.025000
6	0.975000	5Y rolling	P component MC	long	9633	240.825000	238	0.024707	0.025000
7	0.975000	5Y rolling	P component MC	short	9633	240.825000	169	0.017544	0.025000

Notably, at the 97.5% VaR level, the expected exception rate is 2.5%. With 9,633 usable dates, the expected exception count is: $0.025 * 9633 = 240.825$. Here, we can see that the 5-year rolling model looks better for long AAPL and long P. AAPL long has 238 exceptions versus 240.825 expected, and direct P long has 250 exceptions, which is also close. This suggests that some of the 99% VaR failure is concentrated in the more extreme tail.

However, CAT remains poorly calibrated. CAT long and CAT short both produce too many exceptions at 97.5%, so the GBM model continues to underestimate CAT risk even at the less extreme confidence level. Short still remains mostly conservative with AAPL short, direct P short, and MC P short all produce fewer than expected.

So therefore, at the 97.5%, the model is much closer for AAPL and P, but CAT still produces too many exceptions. Resultingly, we can see that the model performance depends strongly on asset, position direction and confidence level.

2. Backtesting, exponential equivalent

Repeat the previous problem using a lambda of 0.9989 (a 5 year window equivalent lambda).

How do the results using an exponential weighting compare to the prior results?

Now, instead of estimating the GBM mean and variance using an equal-weighted rolling 5-year window, we use the course exponential-weighting formula with $\lambda = 0.9989$.

For each daily log return series a_t , the exponentially weighted mean is updated by

$$m_N = (1 - \lambda)a_N + \lambda m_{N-1}.$$

The exponentially weighted second moment, which we use to calculate the variance is given as:

$$r_N = (1 - \lambda)a_N^2 + \lambda r_{N-1}.$$

The variance and standard deviation are then

$$v_N = r_N - m_N^2,$$

$$\sigma_N = \sqrt{v_N}.$$

The fundamental idea from this is that the weighted estimates are then inserted into the same 5-day GBM VaR formulas that were utilised for the previous questions. Like before, an exception occurs when

$$L_{t,t+5} > \text{VaR}_t.$$

For the portfolio, the corrected fixed-share setup is

$$P_t = 24679A_t + 171C_t.$$

The direct P method treats P_t as one portfolio value series and applies EWMA to the log returns of P . The component Monte Carlo method instead estimates EWMA means, variances, and covariance for AAPL and CAT separately, then reprices the fixed-share portfolio under simulated AAPL/CAT return shocks:

$$P_{t+5}^{(j)} = 24679A_te^{R_A^{(j)}} + 171C_te^{R_C^{(j)}}.$$

For the component covariance, the cross moment is updated as

$$r_N^{AC} = (1 - \lambda)a_Nc_N + \lambda r_{N-1}^{AC},$$

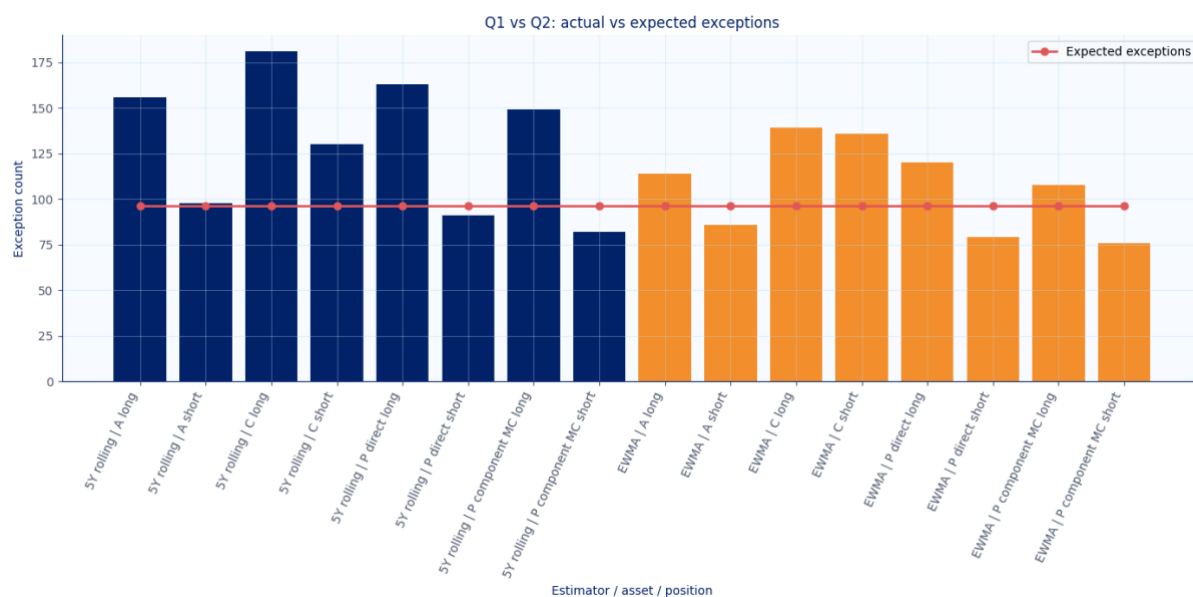
so

$$\text{COV}_{AC,N} = r_N^{AC} - m_N^A m_N^C.$$

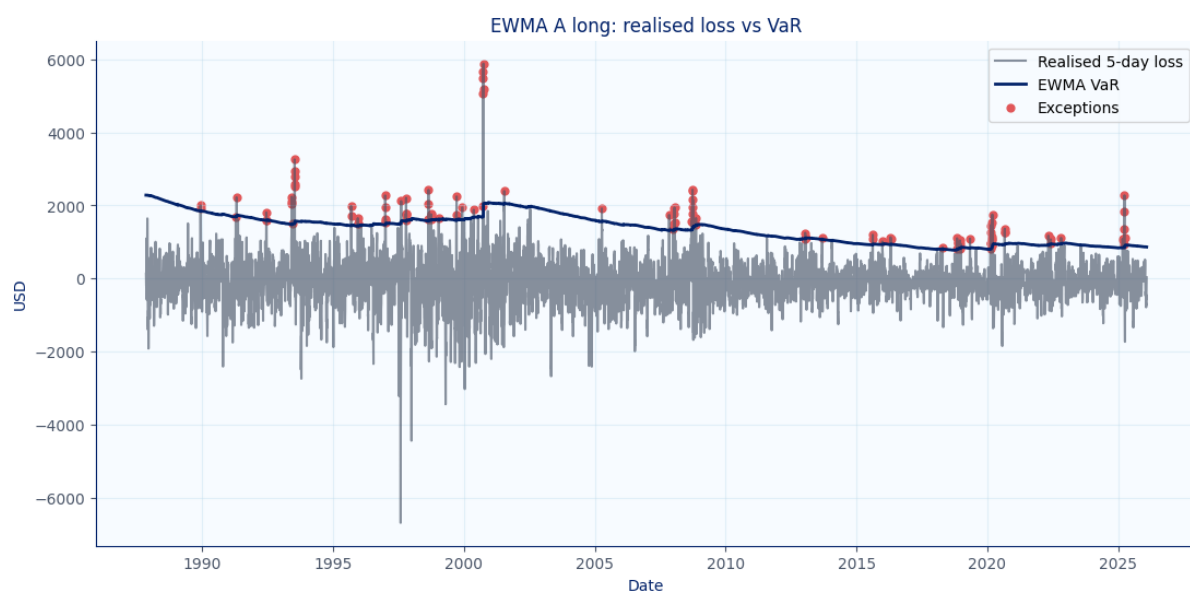
The idea is that by modelling the portfolio individually, or as an addition of two assets, we may arrive at different values (we do). Like before, we have 96.33 as the expect number of exceptions.

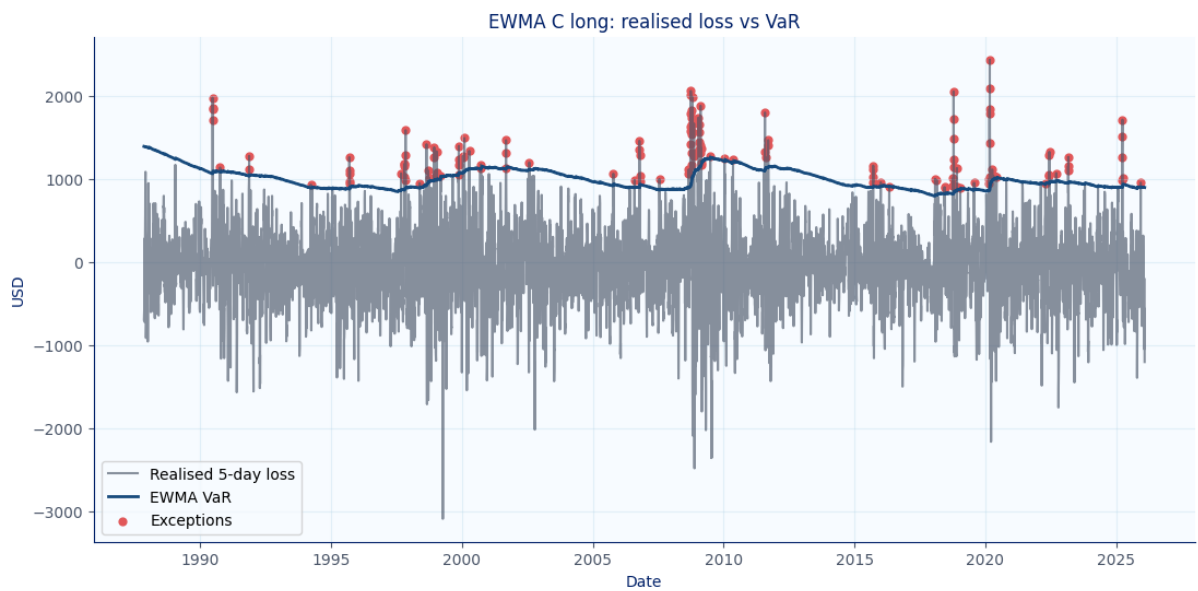
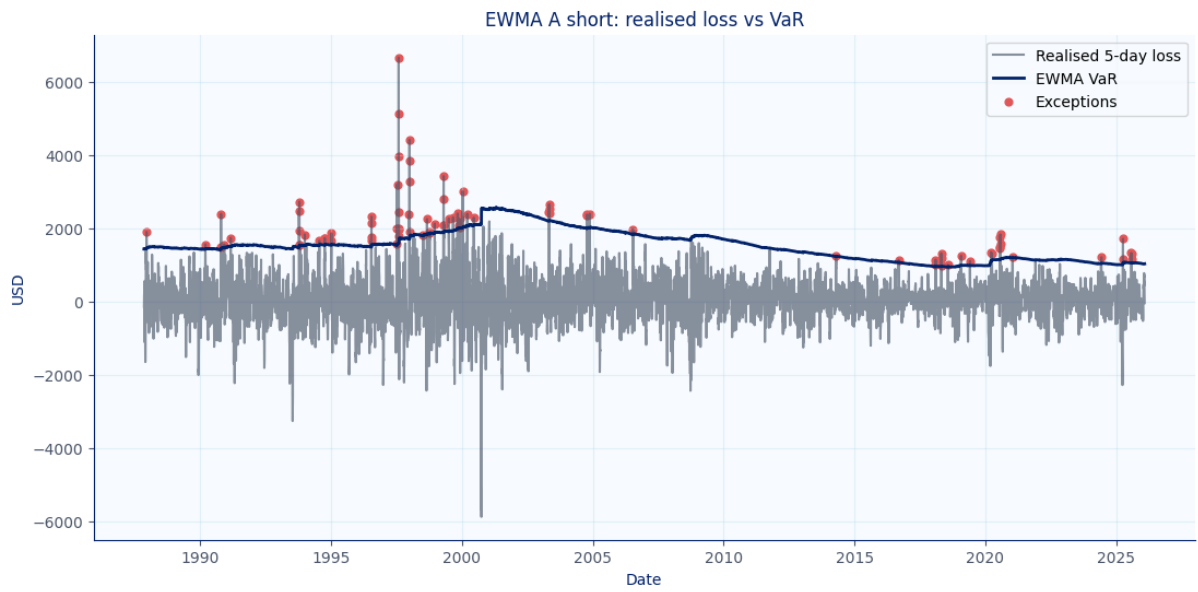
	Estimator	Asset / method	Position	Backtest dates	Expected exceptions	Actual exceptions	Exception rate	Expected exception rate
0	5Y rolling	A	long	9633	96.330000	156	0.016194	0.010000
1	5Y rolling	A	short	9633	96.330000	98	0.010173	0.010000
2	5Y rolling	C	long	9633	96.330000	181	0.018790	0.010000
3	5Y rolling	C	short	9633	96.330000	130	0.013495	0.010000
4	5Y rolling	P direct	long	9633	96.330000	163	0.016921	0.010000
5	5Y rolling	P direct	short	9633	96.330000	91	0.009447	0.010000
6	5Y rolling	P component MC	long	9633	96.330000	149	0.015468	0.010000
7	5Y rolling	P component MC	short	9633	96.330000	82	0.008512	0.010000
8	EWMA lambda=0.9989	A	long	9633	96.330000	114	0.011834	0.010000
9	EWMA lambda=0.9989	A	short	9633	96.330000	86	0.008928	0.010000
10	EWMA lambda=0.9989	C	long	9633	96.330000	139	0.014430	0.010000
11	EWMA lambda=0.9989	C	short	9633	96.330000	136	0.014118	0.010000
12	EWMA lambda=0.9989	P direct	long	9633	96.330000	120	0.012457	0.010000
13	EWMA lambda=0.9989	P direct	short	9633	96.330000	79	0.008201	0.010000
14	EWMA lambda=0.9989	P component MC	long	9633	96.330000	108	0.011211	0.010000
15	EWMA lambda=0.9989	P component MC	short	9633	96.330000	76	0.007890	0.010000

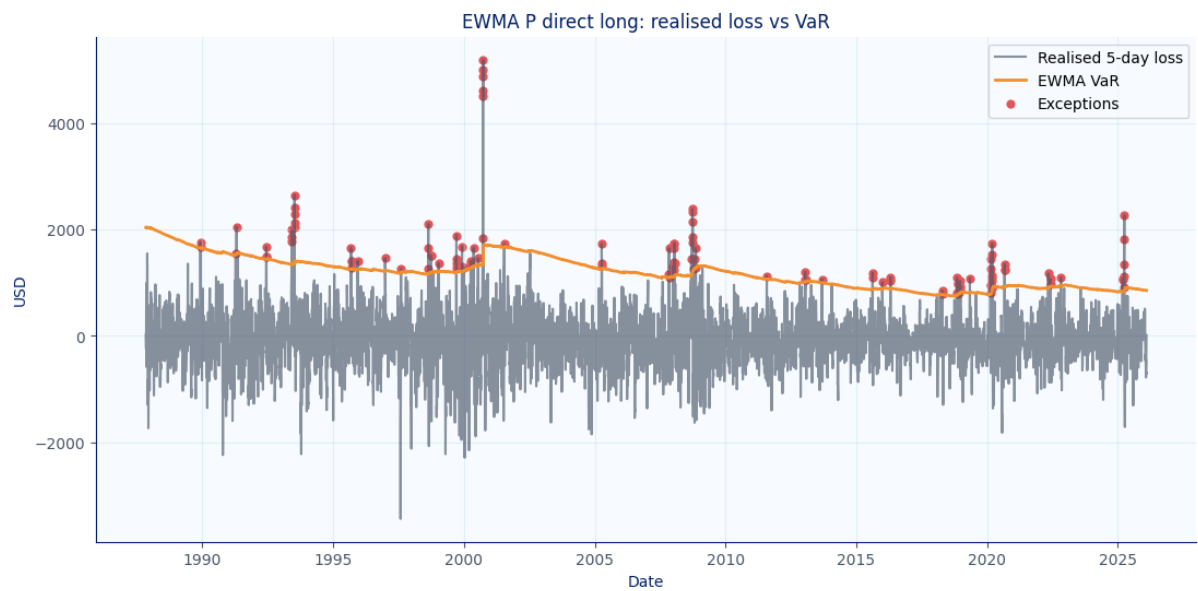
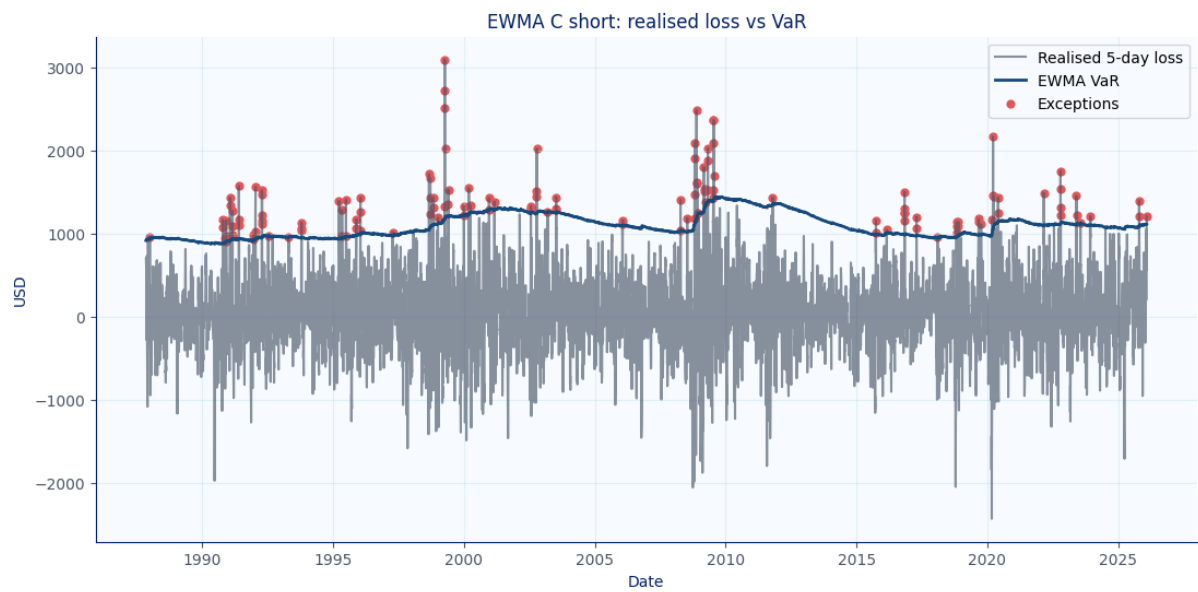
Compared with the 5-year equal-weighted method, EWMA reduces the number of long-position exceptions materially, as we can see below. We should know that the % will be basically the same, as they have the same number of dates measure:

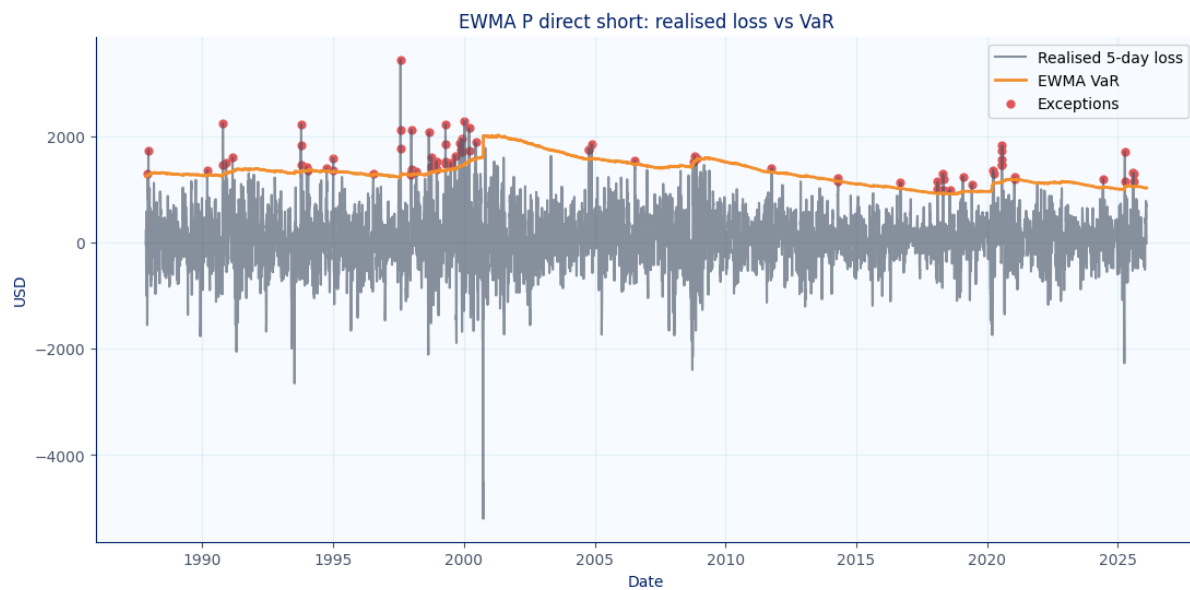


The EWMA estimator improves the long-position VaR calibration. In Q1, the long-position exception counts were substantially above the expected value of 96.33. Under EWMA, they are still above expectation, but much closer. For example, direct P long falls from 163 exceptions to 120, and component Monte Carlo P long falls from 149 to 108. This is likely due to the short-memory feature of EWMA, wherein it is more sensitive to recent volatility, and less sensitive to previous shocks.

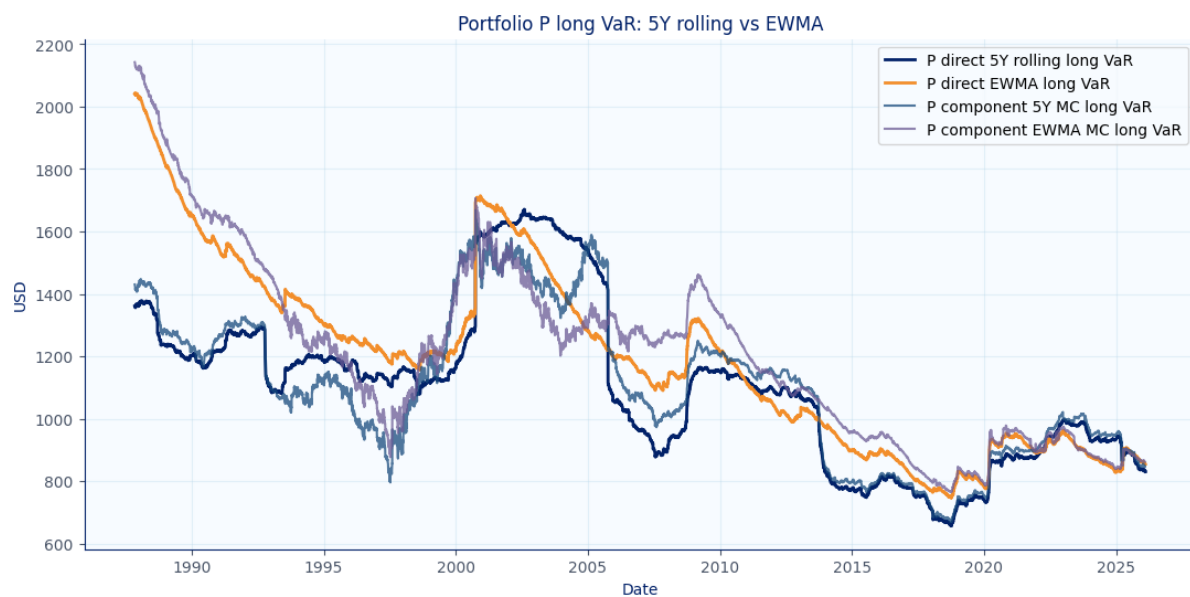


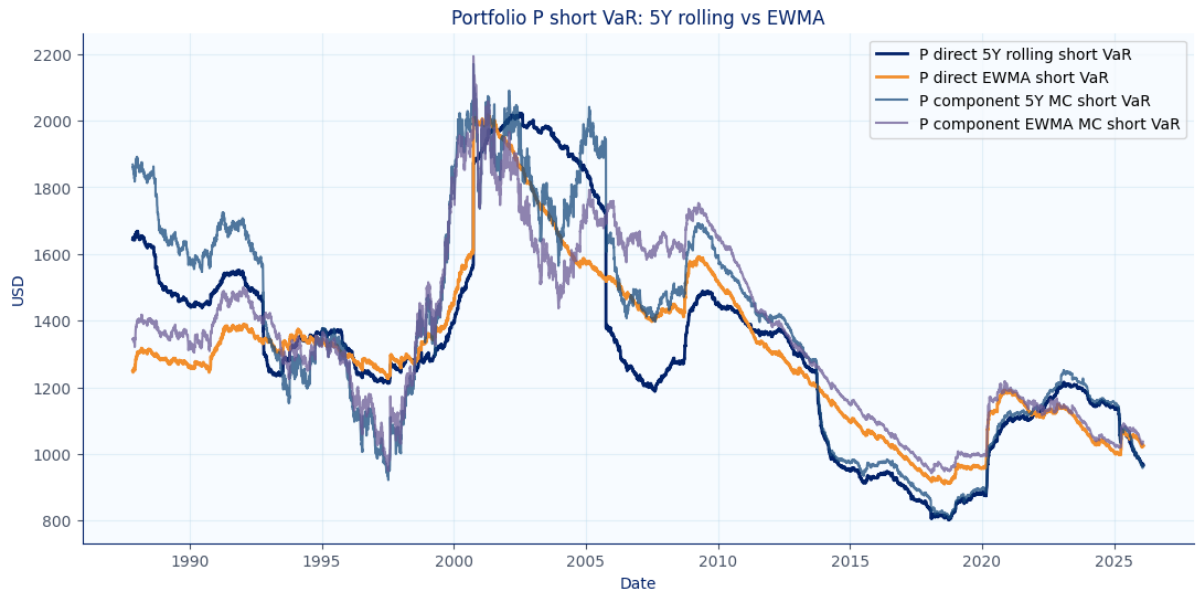






The realised-loss-versus-VaR plots show that EWMA reacts more smoothly to changing volatility. This is the main advantage of exponential weighting, due to once again, *short memory*. The 5-year rolling window gives equal weight to all observations inside the window and zero weight to observations outside it. EWMA instead continuously downweights older observations, so the risk estimate responds more naturally when volatility regimes change. The idea is that conversely, EWMA may be more volatile during periods of shock.





We can directly see the above impacts, for the long portfolio VaR figure, EWMA is generally smoother and more persistent than the 5-year rolling estimate, which helps explain why EWMA reduces long-portfolio exceptions relative to the rolling-window method.

Conversely, for the short portfolio VaR figure, the direct-P and component Monte Carlo curves follow similar broad regimes, but differ materially in some periods, like during the GFC, and the pre-Covid period, showing that modelling the fixed-share portfolio directly as one GBM is not equivalent to repricing AAPL and CAT jointly.

Using $\lambda = 0.9989$, the EWMA VaR estimates generally perform better than the 5-year equal-weighted estimates, especially for long positions.

Asset / method	Position	5Y rolling exceptions	EWMA exceptions	Change
AAPL	Long	156	120	Fewer exceptions
AAPL	Short	98	90	Fewer exceptions
CAT	Long	181	139	Fewer exceptions
CAT	Short	130	136	More exceptions
P direct	Long	163	130	Fewer exceptions
P direct	Short	91	79	Fewer exceptions

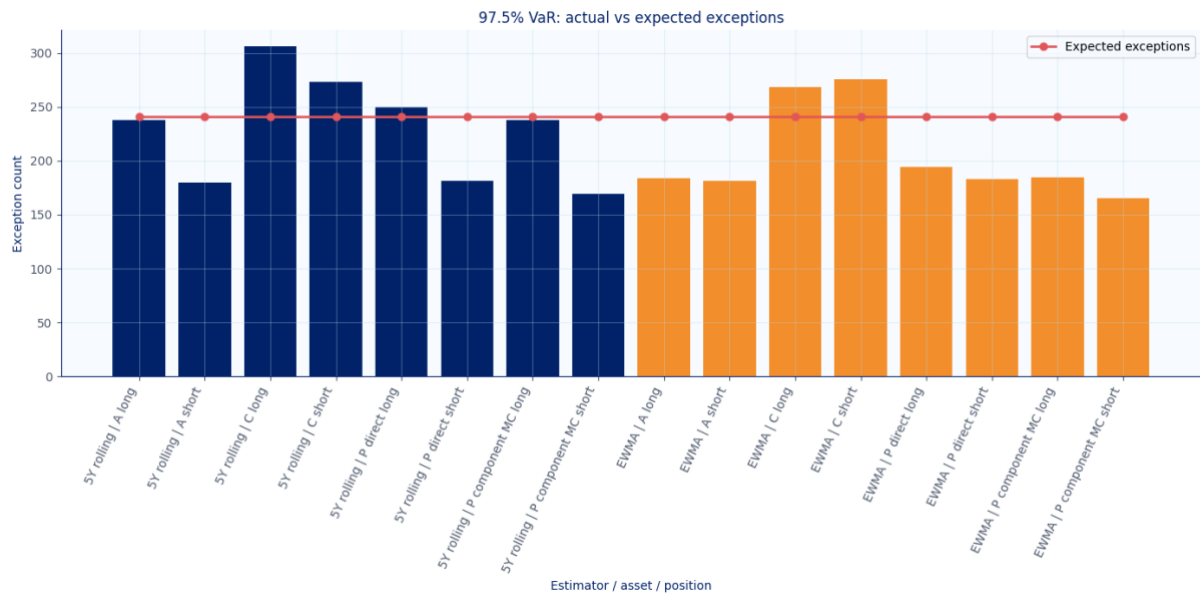
P component MC	Long	149	108	Fewer exceptions
P component MC	Short	82	76	Fewer exceptions

The expected number of exceptions is 96.33. Under EWMA, AAPL long has 114 exceptions, CAT long has 139, direct P long has 120, and component Monte Carlo P long has 108. These are still above the expected number, but they are much closer than the Q1 equal-weighted results. We note the above, that basically across all figures, EWMA generally improves calibration by reducing excessive long-position exceptions, but CAT short gets slightly worse, increasing from 130 to 136 exceptions. This can potentially suggest that the GBM VaR model still misses tail clustering and regime-dependent volatility.

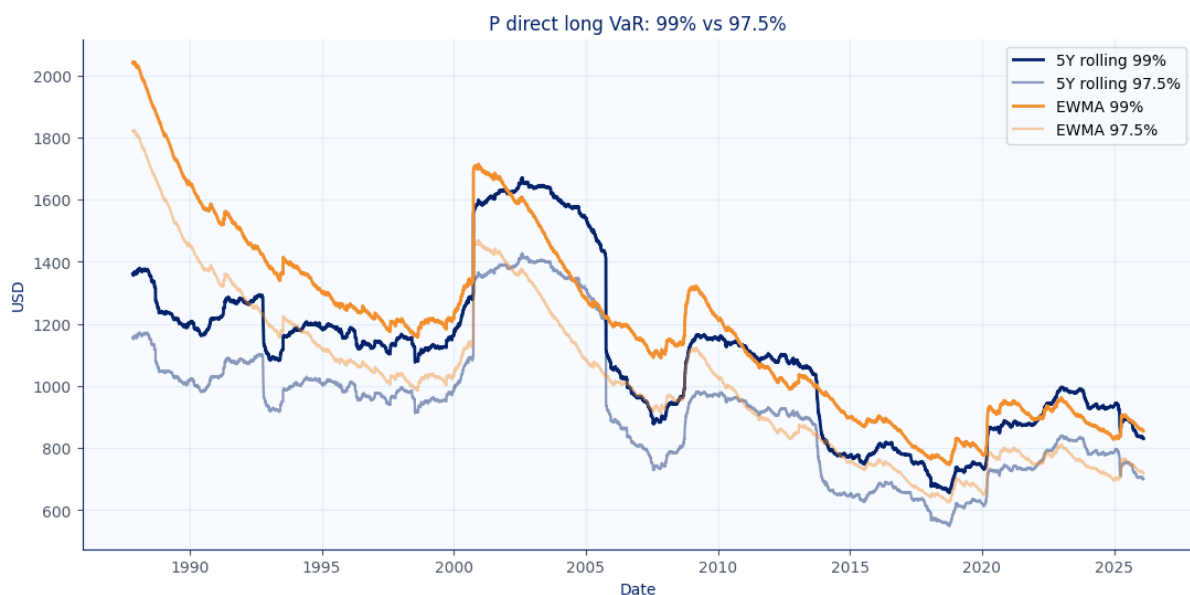
We note the 97.5% vs 99% overall:

99% vs 97.5% comparison table										
	VaR level	Estimator	Asset / method	Position	Backtest dates	Expected exceptions	Actual exceptions	Exception rate (%)	Expected exception rate (%)	
0	0.990000	5Y rolling	A	long	9633	96.330000	156	1.619000	1.000000	
1	0.990000	5Y rolling	A	short	9633	96.330000	98	1.017000	1.000000	
2	0.990000	5Y rolling	C	long	9633	96.330000	181	1.879000	1.000000	
3	0.990000	5Y rolling	C	short	9633	96.330000	130	1.350000	1.000000	
4	0.990000	5Y rolling	P direct	long	9633	96.330000	163	1.692000	1.000000	
5	0.990000	5Y rolling	P direct	short	9633	96.330000	91	0.945000	1.000000	
6	0.990000	5Y rolling	P component MC	long	9633	96.330000	149	1.547000	1.000000	
7	0.990000	5Y rolling	P component MC	short	9633	96.330000	82	0.851000	1.000000	
8	0.990000	EWMA lambda=0.9989	A	long	9633	96.330000	114	1.183000	1.000000	
9	0.990000	EWMA lambda=0.9989	A	short	9633	96.330000	86	0.893000	1.000000	
10	0.990000	EWMA lambda=0.9989	C	long	9633	96.330000	139	1.443000	1.000000	
11	0.990000	EWMA lambda=0.9989	C	short	9633	96.330000	136	1.412000	1.000000	
12	0.990000	EWMA lambda=0.9989	P direct	long	9633	96.330000	120	1.246000	1.000000	
13	0.990000	EWMA lambda=0.9989	P direct	short	9633	96.330000	79	0.820000	1.000000	
14	0.990000	EWMA lambda=0.9989	P component MC	long	9633	96.330000	108	1.121000	1.000000	
15	0.990000	EWMA lambda=0.9989	P component MC	short	9633	96.330000	76	0.789000	1.000000	
16	0.975000	5Y rolling	A	long	9633	240.830000	238	2.471000	2.500000	
17	0.975000	5Y rolling	A	short	9633	240.830000	180	1.869000	2.500000	
18	0.975000	5Y rolling	C	long	9633	240.830000	306	3.177000	2.500000	
19	0.975000	5Y rolling	C	short	9633	240.830000	273	2.834000	2.500000	
20	0.975000	5Y rolling	P direct	long	9633	240.830000	250	2.595000	2.500000	
21	0.975000	5Y rolling	P direct	short	9633	240.830000	181	1.879000	2.500000	
22	0.975000	5Y rolling	P component MC	long	9633	240.830000	238	2.471000	2.500000	
23	0.975000	5Y rolling	P component MC	short	9633	240.830000	169	1.754000	2.500000	
24	0.975000	EWMA lambda=0.9989	A	long	9633	240.830000	184	1.910000	2.500000	
25	0.975000	EWMA lambda=0.9989	A	short	9633	240.830000	181	1.879000	2.500000	
26	0.975000	EWMA lambda=0.9989	C	long	9633	240.830000	268	2.782000	2.500000	
27	0.975000	EWMA lambda=0.9989	C	short	9633	240.830000	276	2.865000	2.500000	
28	0.975000	EWMA lambda=0.9989	P direct	long	9633	240.830000	194	2.014000	2.500000	
29	0.975000	EWMA lambda=0.9989	P direct	short	9633	240.830000	183	1.900000	2.500000	
30	0.975000	EWMA lambda=0.9989	P component MC	long	9633	240.830000	185	1.920000	2.500000	
31	0.975000	EWMA lambda=0.9989	P component MC	short	9633	240.830000	165	1.713000	2.500000	

And more specifically:



In general, we can see that at 97.5%, EWMA is mostly conservative. AAPL long and short both produce fewer exceptions than expected. Direct P and component Monte Carlo P are also conservative for both long and short positions. CAT is again the main exception, both CAT long and CAT short produce more exceptions than expected, so EWMA does not fully fix CAT's tail-risk underestimation. In fact, EWMA or calibrating the confidence fails to change much.



It is good news that the model is not uniformly underestimating risk in the 97.5% case, as was for the 99% case,. For the AAPL and P, the EWMA is generally too high, but for CAT, irrespective of anything, it still underestimates.

EWMA improves the 99% backtest relative to the 5-year rolling method, especially for long positions. However, at 97.5%, EWMA becomes conservative for most assets and portfolio methods. Overall, EWMA helps with volatility adaptation (as it is more sensitive to recent

shocks), but it does not eliminate GBM model risk, especially for assets with stronger tail behaviour or volatility clustering.

HW11 Q1 to Q2: GBM VaR Backtesting

Columbia University · MATH GR 5320 · Financial Risk Management and Regulation

AAPL, CAT, and fixed-share portfolio $P_t = 24679A_t + 171C_t$.

This notebook contains both required HW10 pieces:

1. **Q1:** backtest the 5-year rolling-window GBM VaR estimates for long and short portfolios.
2. **Q2:** repeat the same backtest using the course exponential-weighting recursion with $\lambda = 0.9989$.

1. Backtest logic

For each date t , use the previous five years of daily log returns to estimate GBM parameters. Then compare the VaR estimated on date t with the realised 5-day loss from t to $t + 5$.

The daily log return is

$$a_t = \log\left(\frac{S_t}{S_{t-1}}\right).$$

The future 5-day log return is

$$R_{t,t+5} = \log\left(\frac{S_{t+5}}{S_t}\right).$$

For a long USD 10,000 position, the realised loss is

$$L_{t,t+5}^{\text{long}} = 10000 (1 - e^{R_{t,t+5}}).$$

For a short USD 10,000 position, the realised loss is

$$L_{t,t+5}^{\text{short}} = 10000 (e^{R_{t,t+5}} - 1).$$

An exception occurs when

$$L_{t,t+5} > \text{VaR}_t.$$

2. GBM VaR formula used for A, C, and direct P

For each asset or portfolio value series, assume the 5-day log return is approximately normal:

$$R_h \sim N(h\mu, h\sigma^2), \quad h = 5.$$

The long-position VaR at confidence level q is

$$\text{VaR}_{q,t}^{\text{long}} = V_0 \left(1 - \exp \left[h \hat{\mu}_t + \sqrt{h} \hat{\sigma}_t N^{-1}(1 - q) \right] \right).$$

The short-position VaR is

$$\text{VaR}_{q,t}^{\text{short}} = V_0 \left(\exp \left[h \hat{\mu}_t + \sqrt{h} \hat{\sigma}_t N^{-1}(q) \right] - 1 \right).$$

This is the correct sign convention: long positions lose in the left tail; short positions lose in the right tail.

```
In [2]: from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from statistics import NormalDist

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from IPython.display import display, Markdown

In [3]: COLUMBIA_BLUE = '#B9D9EB'
COLUMBIA_NAVY = '#012169'
COLUMBIA_DARK = '#174A7C'
COLUMBIA_GRAY = '#4A5568'
COLUMBIA_ORANGE = '#F28E2B'
COLUMBIA_PURPLE = '#6B5B95'
COLUMBIA_GREEN = '#59A14F'
COLUMBIA_RED = '#E15759'

plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['axes.facecolor'] = '#F7FBFF'
plt.rcParams['figure.facecolor'] = 'white'
plt.rcParams['axes.edgecolor'] = COLUMBIA_NAVY
plt.rcParams['axes.labelcolor'] = COLUMBIA_NAVY
plt.rcParams['axes.titlecolor'] = COLUMBIA_NAVY
plt.rcParams['xtick.color'] = COLUMBIA_GRAY
plt.rcParams['ytick.color'] = COLUMBIA_GRAY
plt.rcParams['grid.color'] = COLUMBIA_BLUE
plt.rcParams['grid.alpha'] = 0.35
plt.rcParams['axes.grid'] = True
plt.rcParams['axes.spines.top'] = False
plt.rcParams['axes.spines.right'] = False
pd.options.display.float_format = '{:,.6f}'.format
NORMAL = NormalDist()

In [4]: @dataclass(frozen=True)
class Config:
    notional: float = 10_000.0
    var_level: float = 0.99
    horizon_days: int = 5
    window_years: int = 5
    trading_days: int = 252
    aapl_shares: float = 24679.0
    cat_shares: float = 171.0
```

```

mc_paths: int = 5_000
mc_seed: int = 5320
ewma_lambda: float = 0.9989

@property
def window_days(self) -> int:
    return self.window_years * self.trading_days

cfg = Config()
cfg

```

Out [4]: Config(notional=10000.0, var_level=0.99, horizon_days=5, window_years=5, trading_days=252, aapl_shares=24679.0, cat_shares=171.0, mc_paths=5000, mc_seed=5320, ewma_lambda=0.9989)

3. Load AAPL and CAT data

The notebook first checks the local `data/` directory. If the files are not found there, it falls back to the uploaded sandbox paths.

```

In [5]: def find_file(names):
    candidates = []
    for name in names:
        candidates.append(Path('data') / name)
        candidates.append(Path('/mnt/data') / name)
    for path in candidates:
        if path.exists():
            return path
    raise FileNotFoundError(names)

def read_bloomberg_file(path: Path, ticker: str) -> pd.DataFrame:
    if path.suffix.lower() == '.csv':
        raw = pd.read_csv(path)
        if 'Dates' not in raw.columns and len(raw) > 0:
            raw.columns = raw.iloc[0].tolist()
            raw = raw.iloc[1:].copy()
    else:
        raw = pd.read_excel(path)
        if 'Dates' not in raw.columns:
            header_row = raw.index[raw.iloc[:, 0].astype(str).str.lower()]
            if len(header_row) > 0:
                i = int(header_row[0])
                raw.columns = raw.iloc[i].tolist()
                raw = raw.iloc[i + 1:].copy()
        raw.columns = [str(c).strip() for c in raw.columns]
        date_col = 'Dates' if 'Dates' in raw.columns else 'Date'
        out = raw[[date_col, 'PX_LAST']].copy()
        out.columns = ['Date', ticker]
        out['Date'] = pd.to_datetime(out['Date'])
        out[ticker] = pd.to_numeric(out[ticker], errors='coerce')
        return out.dropna().sort_values('Date').reset_index(drop=True)

aapl_path = find_file(['AAPL-bloomberg.csv', 'AAPL-bloomberg.xlsx'])
cat_path = find_file(['CAT-bloomberg.csv', 'CAT-bloomberg.xlsx'])

a = read_bloomberg_file(aapl_path, 'A')

```

```

c = read_bloomberg_file(cat_path, 'C')

df = a.merge(c, on='Date', how='inner').sort_values('Date').reset_index()
df['P'] = cfg.aapl_shares * df['A'] + cfg.cat_shares * df['C']

for col in ['A', 'C', 'P']:
    df[f'r_{col}'] = np.log(df[col] / df[col].shift(1))
    df[f'future_{col}'] = np.log(df[col].shift(-cfg.horizon_days) / df[col])
    df[f'loss_long_{col}'] = cfg.notional * (1 - np.exp(df[f'future_{col}']))
    df[f'loss_short_{col}'] = cfg.notional * (np.exp(df[f'future_{col}']))

display(Markdown(f'Loaded AAPL from `{aapl_path}` and CAT from `{cat_path}`'))
display(df.head())
display(df.tail())

```

Loaded AAPL from `data/AAPL-bloomberg.csv` and CAT from `data/CAT-bloomberg.csv`.

	Date	A	C	P	r_A	future_A	loss_long_A	loss
0	1982-11-15	0.141200	5.500000	4,425.174800	NaN	-0.082643	793.201133	-
1	1982-11-16	0.133900	5.359400	4,220.975500	-0.053084	-0.038056	373.412995	-3
2	1982-11-17	0.140100	5.281300	4,360.630200	0.045263	-0.061830	599.571734	-4
3	1982-11-18	0.140100	5.312500	4,365.965400	0.000000	-0.078676	756.602427	-7
4	1982-11-19	0.137800	5.312500	4,309.203700	-0.016553	-0.066766	645.863570	-6

	Date	A	C	P	r_A	future_A	loss
10893	2026-02-05	275.910000	678.310000	6,925,173.900000	-0.002100	NaN	
10894	2026-02-06	278.120000	726.200000	6,987,903.680000	0.007978	NaN	
10895	2026-02-09	274.620000	742.120000	6,904,249.500000	-0.012664	NaN	
10896	2026-02-10	273.680000	742.370000	6,881,093.990000	-0.003429	NaN	
10897	2026-02-11	275.500000	775.000000	6,931,589.500000	0.006628	NaN	

```

In [6]: fig, ax = plt.subplots(figsize=(13, 6))
        base = df[['A', 'C', 'P']].iloc[0]
        ax.plot(df['Date'], 100 * df['A'] / base['A'], label='AAPL indexed to 100')
        ax.plot(df['Date'], 100 * df['C'] / base['C'], label='CAT indexed to 100')
        ax.plot(df['Date'], 100 * df['P'] / base['P'], label='Portfolio P indexed to 100')
        ax.set_title('AAPL, CAT, and fixed-share portfolio P')
        ax.set_xlabel('Date')

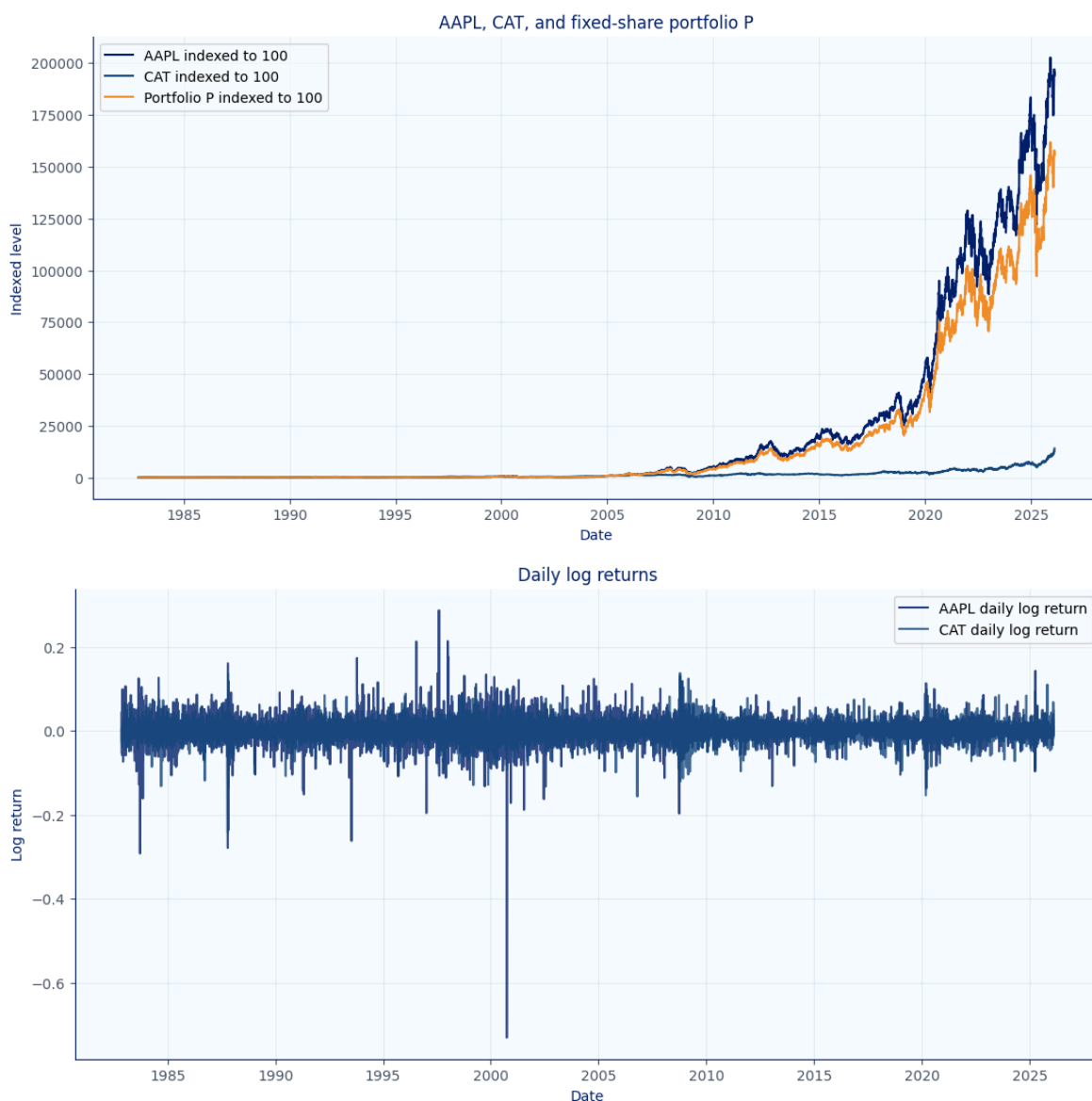
```

```

ax.set_ylabel('Indexed level')
ax.legend()
ax.xaxis.set_major_locator(mdates.YearLocator(5))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.show()

fig, ax = plt.subplots(figsize=(13, 6))
ax.plot(df['Date'], df['r_A'], label='AAPL daily log return', color=COLUM
ax.plot(df['Date'], df['r_C'], label='CAT daily log return', color=COLUMB
ax.set_title('Daily log returns')
ax.set_xlabel('Date')
ax.set_ylabel('Log return')
ax.legend()
ax.xaxis.set_major_locator(mdates.YearLocator(5))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.show()

```



4. Rolling 5-year GBM VaR

For A, C, and direct P, the notebook estimates a rolling daily mean and variance using the previous 1,260 trading days.

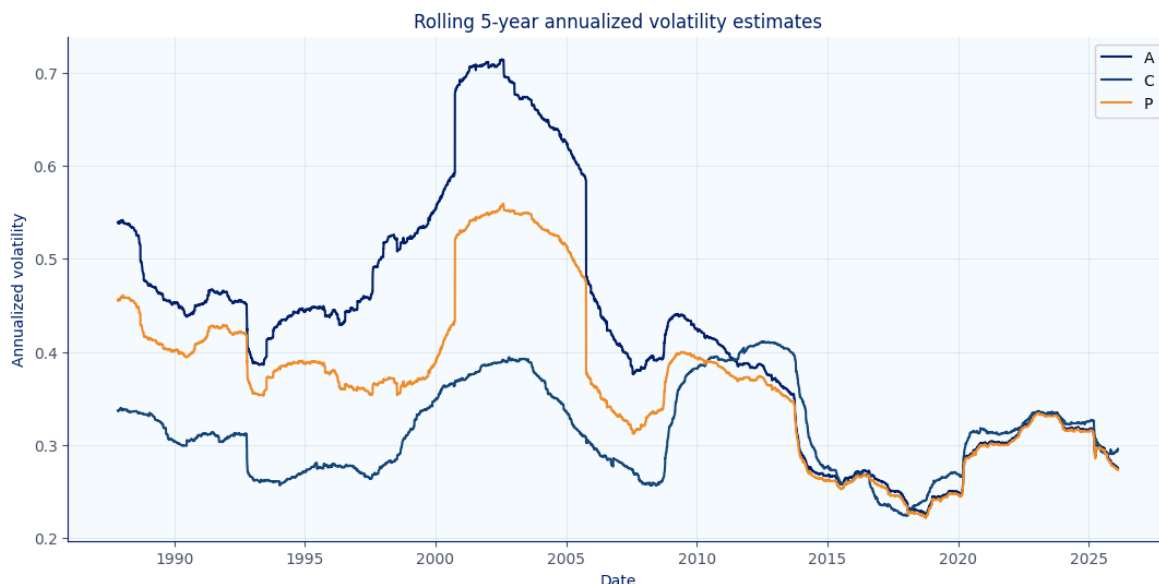
```
In [7]: def add_direct_gbm_var(data: pd.DataFrame, label: str, cfg: Config) -> pd
out = data.copy()
ret = out[f'r_{label}']
mu = ret.rolling(cfg.window_days).mean()
var = ret.rolling(cfg.window_days).var(ddof=1)
sig_h = np.sqrt(cfg.horizon_days * var)
mu_h = cfg.horizon_days * mu
z_left = NORMAL.inv_cdf(1 - cfg.var_level)
z_right = NORMAL.inv_cdf(cfg.var_level)
out[f'mu_{label}'] = mu
out[f'var_daily_{label}'] = var
out[f'vol_daily_{label}'] = np.sqrt(var)
out[f'VaR_long_{label}'] = cfg.notional * (1 - np.exp(mu_h + sig_h *
out[f'VaR_short_{label}'] = cfg.notional * (np.exp(mu_h + sig_h * z_r
out[f'exc_long_{label}'] = out[f'VaR_long_{label}'].notna() & out[f'f
out[f'exc_short_{label}'] = out[f'VaR_short_{label}'].notna() & out[f
return out

for label in ['A', 'C', 'P']:
    df = add_direct_gbm_var(df, label, cfg)

df['year'] = df['Date'].dt.year
display(df[['Date', 'A', 'C', 'P', 'VaR_long_A', 'VaR_short_A', 'VaR_long
```

	Date	A	C	P	VaR_long_A	VaR_short_A
10893	2026-02-05	275.910000	678.310000	6,925,173.900000	835.869472	976.17434'
10894	2026-02-06	278.120000	726.200000	6,987,903.680000	835.962266	975.553127
10895	2026-02-09	274.620000	742.120000	6,904,249.500000	836.807696	974.912353
10896	2026-02-10	273.680000	742.370000	6,881,093.990000	836.589292	975.032278
10897	2026-02-11	275.500000	775.000000	6,931,589.500000	836.637656	973.452462

```
In [8]: fig, ax = plt.subplots(figsize=(13, 6))
for label, color in [('A', COLUMBIA_NAVY), ('C', COLUMBIA_DARK), ('P', CO
ax.plot(df['Date'], df[f'vol_daily_{label}'] * np.sqrt(cfg.trading_da
ax.set_title('Rolling 5-year annualized volatility estimates')
ax.set_xlabel('Date')
ax.set_ylabel('Annualized volatility')
ax.legend()
ax.xaxis.set_major_locator(mdates.YearLocator(5))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.show()
```



5. Direct P versus component-level Monte Carlo P

HW9/Q9 also asks whether P should be modelled from the historical returns of P directly or by modelling A and C separately.

The direct-P method fits a GBM to the portfolio value series P_t .

The component Monte Carlo method fits a bivariate GBM to (A_t, C_t) and reprices

$$P_{t+5} = 24679A_te^{R_A} + 171C_te^{R_C}.$$

The USD 10,000 investment is represented by scaling the fixed-share portfolio on each date:

$$\text{scale}_t = \frac{10000}{24679A_t + 171C_t}.$$

```
In [9]: def add_component_mc_p_var(data: pd.DataFrame, cfg: Config) -> pd.DataFrame:
    out = data.copy()
    out['mu_A_component'] = out['r_A'].rolling(cfg.window_days).mean()
    out['mu_C_component'] = out['r_C'].rolling(cfg.window_days).mean()
    out['var_A_component'] = out['r_A'].rolling(cfg.window_days).var(ddof=1)
    out['var_C_component'] = out['r_C'].rolling(cfg.window_days).var(ddof=1)
    out['cov_AC_component'] = out['r_A'].rolling(cfg.window_days).cov(out['r_C'], ddof=1)
    rng = np.random.default_rng(cfg.mc_seed)
    z = rng.standard_normal((cfg.mc_paths, 2))
    var_long = np.full(len(out), np.nan)
    var_short = np.full(len(out), np.nan)
    valid = out['mu_A_component'].notna() & out['future_P'].notna()
    for i in np.where(valid.to_numpy())[0]:
        mu = cfg.horizon_days * np.array([out.at[i, 'mu_A_component'], out.at[i, 'mu_C_component']])
        cov = cfg.horizon_days * np.array([
            [out.at[i, 'var_A_component'], out.at[i, 'cov_AC_component']],
            [out.at[i, 'cov_AC_component'], out.at[i, 'var_C_component']]
        ])
        cov = (cov + cov.T) / 2
        vals, vecs = np.linalg.eigh(cov)
        vals = np.maximum(vals, 1e-16)
```



```

transform = vecs @ np.diag(np.sqrt(vals))
shocks = mu + z @ transform.T
p0 = cfg.aapl_shares * out.at[i, 'A'] + cfg.cat_shares * out.at[i, 'C']
scale = cfg.notional / p0
pf = scale * (
    cfg.aapl_shares * out.at[i, 'A'] * np.exp(shocks[:, 0])
    + cfg.cat_shares * out.at[i, 'C'] * np.exp(shocks[:, 1])
)
var_long[i] = np.quantile(cfg.notional - pf, cfg.var_level)
var_short[i] = np.quantile(pf - cfg.notional, cfg.var_level)
out['VaR_long_P_component_MC'] = var_long
out['VaR_short_P_component_MC'] = var_short
out['exc_long_P_component_MC'] = out['VaR_long_P_component_MC'].notna
out['exc_short_P_component_MC'] = out['VaR_short_P_component_MC'].notna
return out

df = add_component_mc_p_var(df, cfg)
display(df[['Date', 'VaR_long_P', 'VaR_long_P_component_MC', 'VaR_short_P', 'VaR_short_P_component_MC']])

```

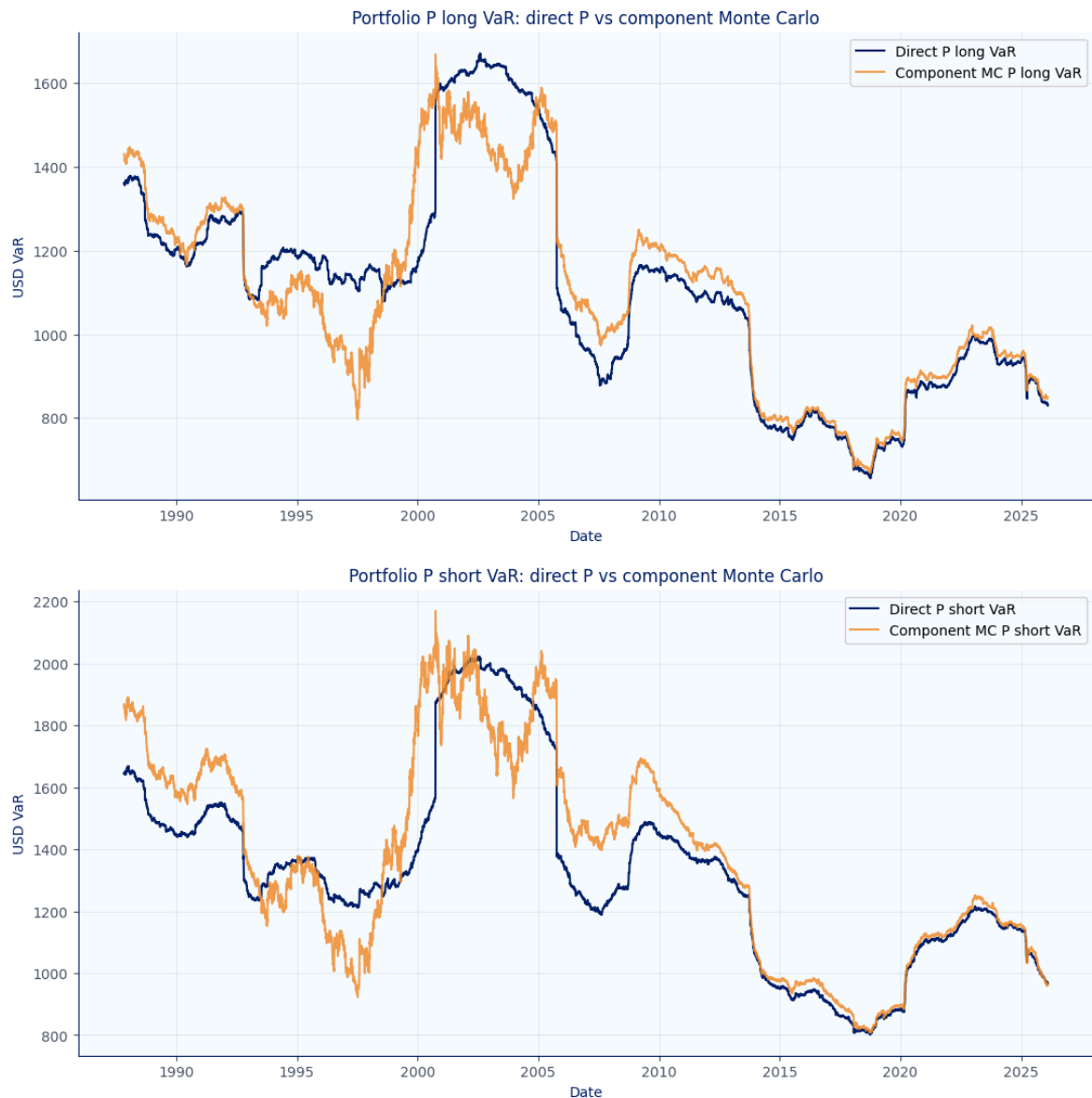
	Date	VaR_long_P	VaR_long_P_component_MC	VaR_short_P	VaR_short_P_component_MC
10893	2026-02-05	829.745760	NaN	969.483344	NaN
10894	2026-02-06	829.817382	NaN	968.932682	NaN
10895	2026-02-09	830.632372	NaN	968.279999	NaN
10896	2026-02-10	830.411459	NaN	968.401068	NaN
10897	2026-02-11	830.444665	NaN	966.884089	NaN

```

In [10]: fig, ax = plt.subplots(figsize=(13, 6))
ax.plot(df['Date'], df['VaR_long_P'], label='Direct P long VaR', color='C0')
ax.plot(df['Date'], df['VaR_long_P_component_MC'], label='Component MC P long VaR', color='C1')
ax.set_title('Portfolio P long VaR: direct P vs component Monte Carlo')
ax.set_xlabel('Date')
ax.set_ylabel('USD VaR')
ax.legend()
ax.xaxis.set_major_locator(mdates.YearLocator(5))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.show()

fig, ax = plt.subplots(figsize=(13, 6))
ax.plot(df['Date'], df['VaR_short_P'], label='Direct P short VaR', color='C0')
ax.plot(df['Date'], df['VaR_short_P_component_MC'], label='Component MC P short VaR', color='C1')
ax.set_title('Portfolio P short VaR: direct P vs component Monte Carlo')
ax.set_xlabel('Date')
ax.set_ylabel('USD VaR')
ax.legend()
ax.xaxis.set_major_locator(mdates.YearLocator(5))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.show()

```



6. Backtest summaries

The expected number of exceptions is

$$(1 - q)N,$$

where N is the number of usable backtest dates. With $q = 0.99$, the expected exception rate is 1%.

```
In [11]: def overall_summary(data: pd.DataFrame, cfg: Config) -> pd.DataFrame:
    specs = [
        ('A', 'long', 'VaR_long_A', 'loss_long_A', 'exc_long_A'),
        ('A', 'short', 'VaR_short_A', 'loss_short_A', 'exc_short_A'),
        ('C', 'long', 'VaR_long_C', 'loss_long_C', 'exc_long_C'),
        ('C', 'short', 'VaR_short_C', 'loss_short_C', 'exc_short_C'),
        ('P direct', 'long', 'VaR_long_P', 'loss_long_P', 'exc_long_P'),
        ('P direct', 'short', 'VaR_short_P', 'loss_short_P', 'exc_short_P'),
        ('P component MC', 'long', 'VaR_long_P_component_MC', 'loss_long_P_component_MC', 'exc_long_P_component_MC'),
        ('P component MC', 'short', 'VaR_short_P_component_MC', 'loss_short_P_component_MC', 'exc_short_P_component_MC')
    ]
    rows = []
    for asset, position, var_col, loss_col, exc_col in specs:
```

```

usable = data[var_col].notna() & data[loss_col].notna()
n = int(usable.sum())
actual = int(data.loc[usable, exc_col].sum())
rows.append({
    'Asset / method': asset,
    'Position': position,
    'Backtest dates': n,
    'Expected exceptions': (1 - cfg.var_level) * n,
    'Actual exceptions': actual,
    'Exception rate': actual / n if n else np.nan,
    'Expected exception rate': 1 - cfg.var_level,
})
return pd.DataFrame(rows)

def annual_summary(data: pd.DataFrame, cfg: Config) -> pd.DataFrame:
    specs = [
        ('A', 'long', 'VaR_long_A', 'loss_long_A', 'exc_long_A'),
        ('A', 'short', 'VaR_short_A', 'loss_short_A', 'exc_short_A'),
        ('C', 'long', 'VaR_long_C', 'loss_long_C', 'exc_long_C'),
        ('C', 'short', 'VaR_short_C', 'loss_short_C', 'exc_short_C'),
        ('P direct', 'long', 'VaR_long_P', 'loss_long_P', 'exc_long_P'),
        ('P direct', 'short', 'VaR_short_P', 'loss_short_P', 'exc_short_P'),
        ('P component MC', 'long', 'VaR_long_P_component_MC', 'loss_long_P_component_MC', 'exc_long_P_component_MC'),
        ('P component MC', 'short', 'VaR_short_P_component_MC', 'loss_short_P_component_MC', 'exc_short_P_component_MC')
    ]
    frames = []
    for asset, position, var_col, loss_col, exc_col in specs:
        usable = data[var_col].notna() & data[loss_col].notna()
        g = data.loc[usable].groupby('year').agg(
            Backtest_dates=(exc_col, 'size'),
            Actual_exceptions=(exc_col, 'sum')
        ).reset_index()
        g['Asset / method'] = asset
        g['Position'] = position
        g['Expected exceptions'] = (1 - cfg.var_level) * g['Backtest_dates']
        g['Exception rate'] = g['Actual_exceptions'] / g['Backtest_dates']
        frames.append(g)
    return pd.concat(frames, ignore_index=True)

overall = overall_summary(df, cfg)
annual = annual_summary(df, cfg)

display(overall)
display(annual.head(20))

```

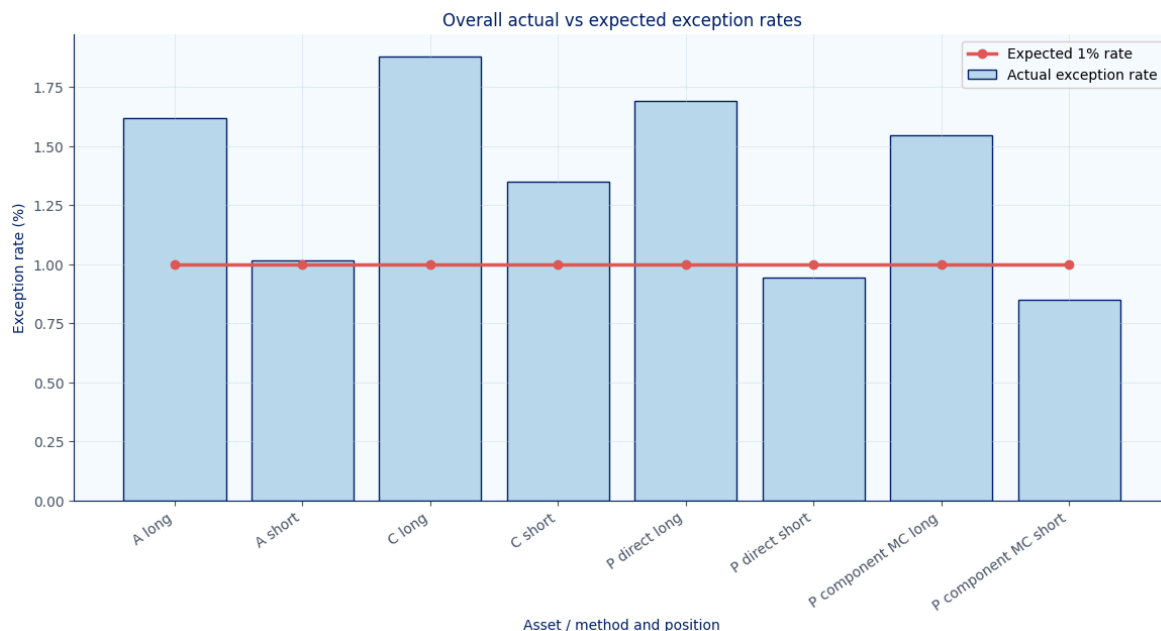
	Asset / method	Position	Backtest dates	Expected exceptions	Actual exceptions	Exception rate	Expected exception rate
0	A	long	9633	96.330000	156	0.016194	0.010000
1	A	short	9633	96.330000	98	0.010173	0.010000
2	C	long	9633	96.330000	181	0.018790	0.010000
3	C	short	9633	96.330000	130	0.013495	0.010000
4	P direct	long	9633	96.330000	163	0.016921	0.010000
5	P direct	short	9633	96.330000	91	0.009447	0.010000
6	P component MC	long	9633	96.330000	149	0.015468	0.010000
7	P component MC	short	9633	96.330000	82	0.008512	0.010000

	year	Backtest_dates	Actual_exceptions	Asset / method	Position	Expected exceptions	Exception rate
0	1987	38	1	A	long	0.380000	0.0263
1	1988	253	0	A	long	2.530000	0.00000
2	1989	252	5	A	long	2.520000	0.0198
3	1990	253	0	A	long	2.530000	0.00000
4	1991	253	8	A	long	2.530000	0.0316
5	1992	254	3	A	long	2.540000	0.0118
6	1993	253	11	A	long	2.530000	0.0434
7	1994	252	0	A	long	2.520000	0.00000
8	1995	252	9	A	long	2.520000	0.0357
9	1996	254	5	A	long	2.540000	0.0196
10	1997	253	8	A	long	2.530000	0.0316
11	1998	252	6	A	long	2.520000	0.0238
12	1999	252	5	A	long	2.520000	0.0198
13	2000	252	7	A	long	2.520000	0.0277
14	2001	248	1	A	long	2.480000	0.0040
15	2002	252	0	A	long	2.520000	0.00000
16	2003	252	0	A	long	2.520000	0.00000
17	2004	252	0	A	long	2.520000	0.00000
18	2005	252	1	A	long	2.520000	0.0039
19	2006	251	0	A	long	2.510000	0.00000

```
In [12]: labels = overall['Asset / method'] + ' ' + overall['Position']
x = np.arange(len(labels))
fig, ax = plt.subplots(figsize=(14, 6))
ax.bar(x, overall['Actual exceptions'], color=COLUMBIA_DARK, label='Actual exceptions')
ax.plot(x, overall['Expected exceptions'], color=COLUMBIA_ORANGE, marker='o')
ax.set_title('Overall actual vs expected VaR exceptions')
ax.set_xlabel('Asset / method and position')
ax.set_ylabel('Exception count')
ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=35, ha='right')
ax.legend()
plt.show()

fig, ax = plt.subplots(figsize=(14, 6))
ax.bar(x, 100 * overall['Exception rate'], color=COLUMBIA_BLUE, edgecolor='black')
ax.plot(x, 100 * overall['Expected exception rate'], color=COLUMBIA_RED, marker='o')
ax.set_title('Overall actual vs expected exception rates')
ax.set_xlabel('Asset / method and position')
ax.set_ylabel('Exception rate (%)')
ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=35, ha='right')
ax.legend()
plt.show()
```

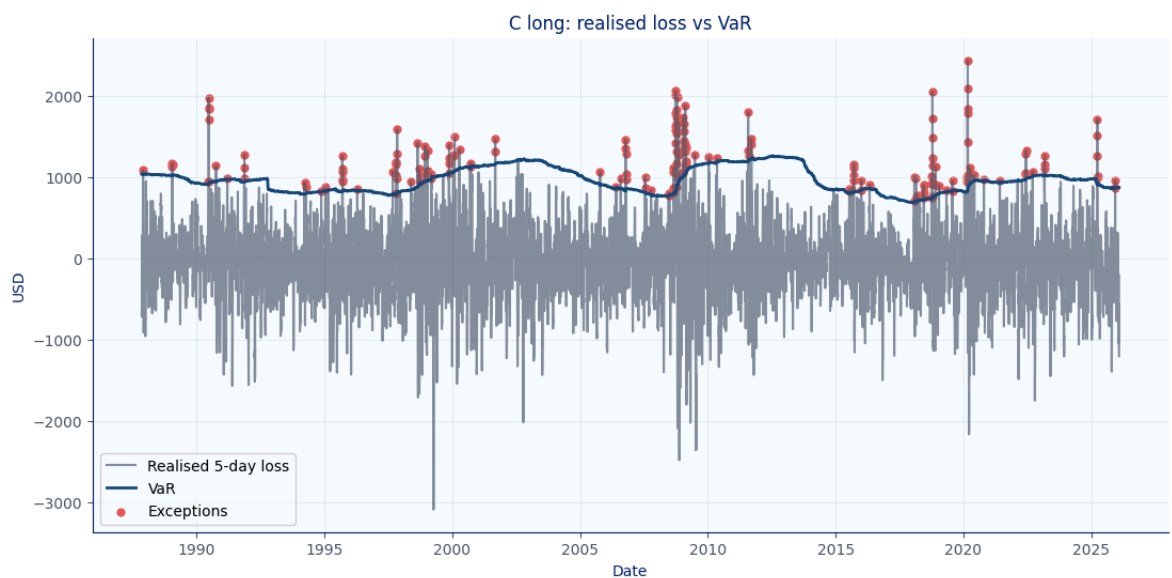
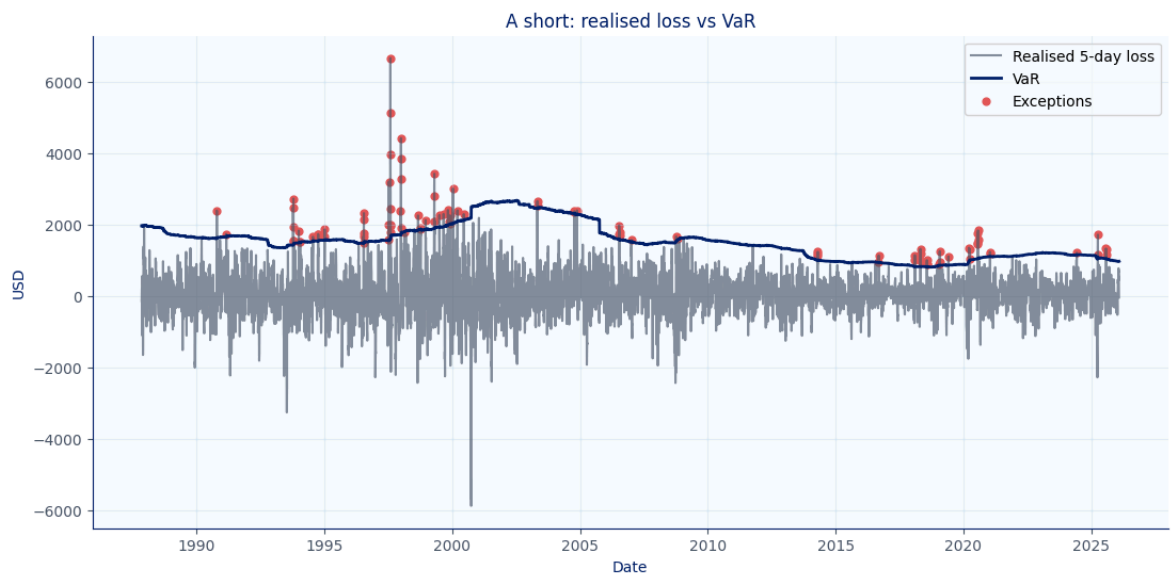
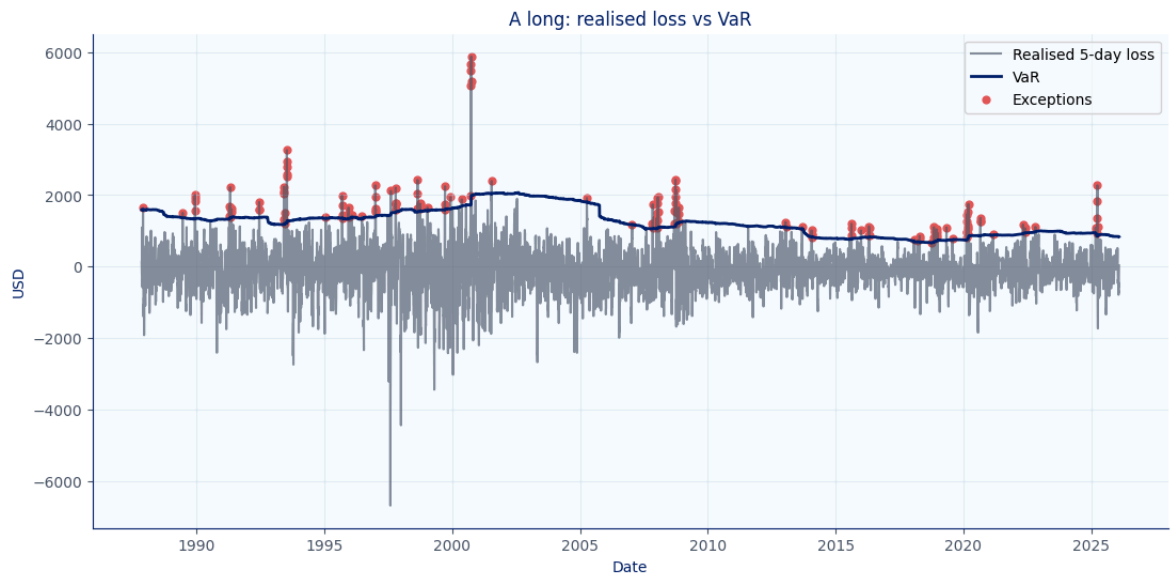


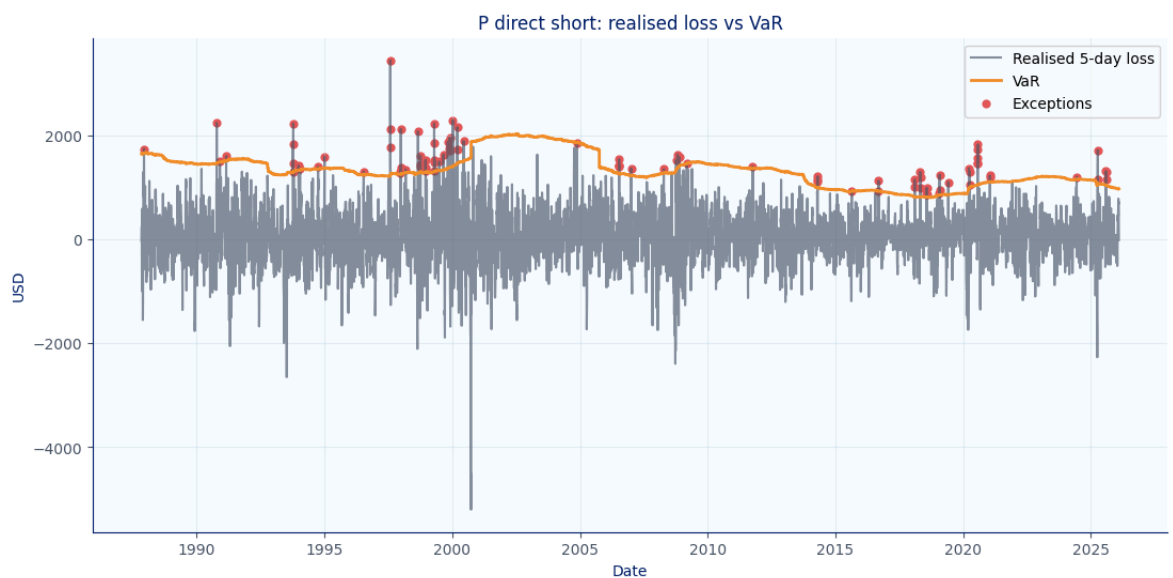
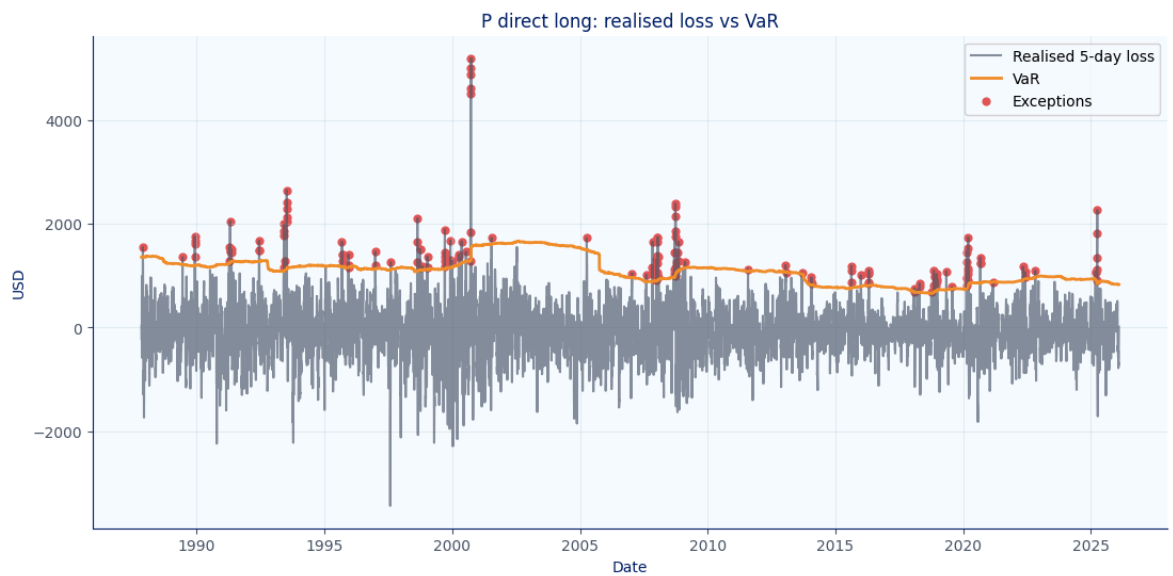
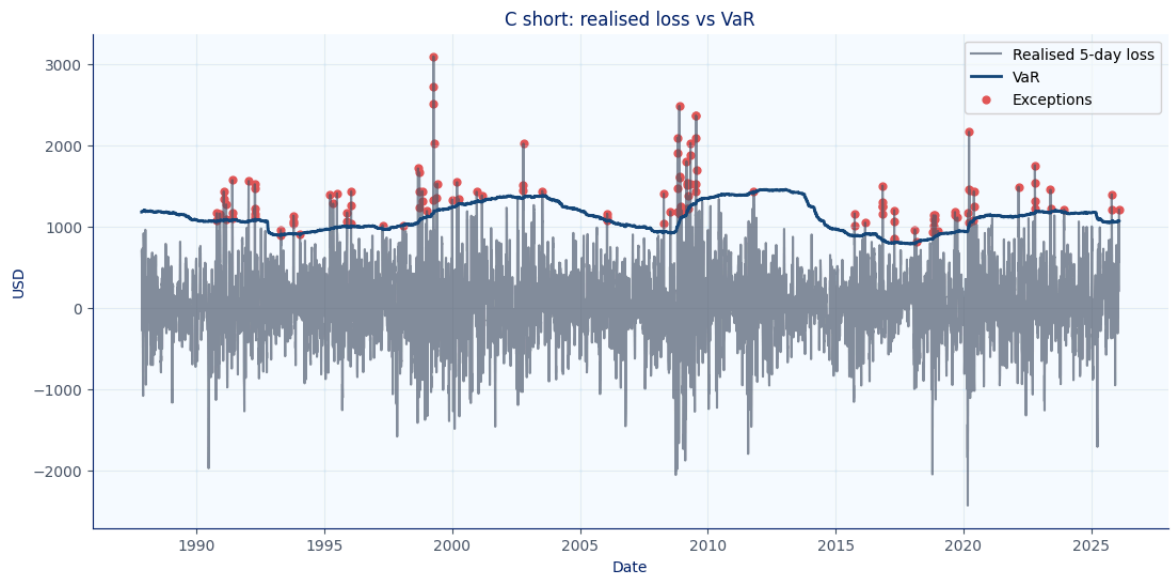


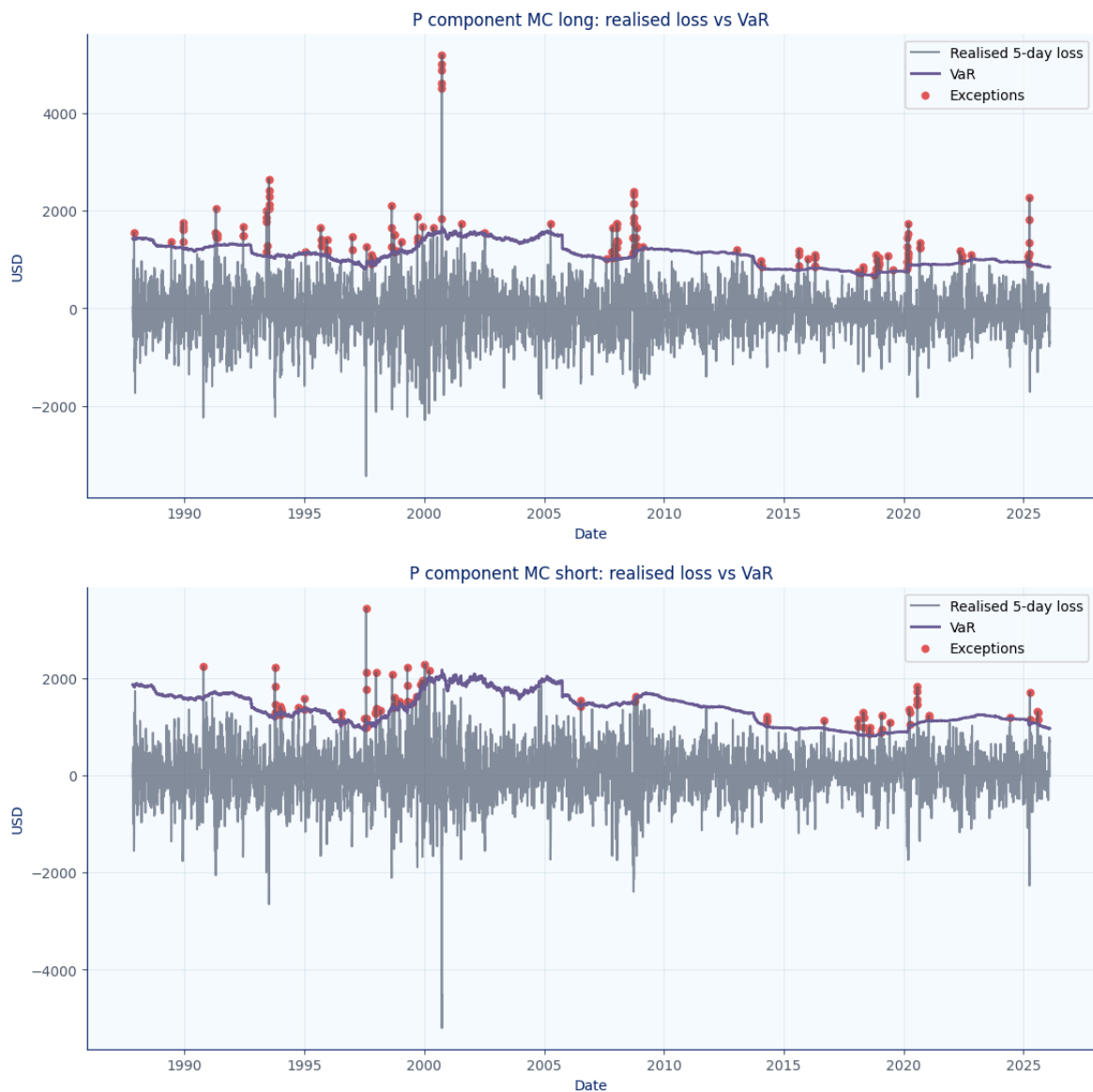
```
In [13]: def plot_loss_vs_var(label, position, var_col, loss_col, exc_col, color):
    usable = df[var_col].notna() & df[loss_col].notna()
    sub = df.loc[usable]
    fig, ax = plt.subplots(figsize=(13, 6))
    ax.plot(sub['Date'], sub[loss_col], color=COLUMBIA_GRAY, alpha=0.65,
            ax.plot(sub['Date'], sub[var_col], color=color, linewidth=2.0, label=
            exc = sub[sub[exc_col]]
    ax.scatter(exc['Date'], exc[loss_col], color=COLUMBIA_RED, s=24, label=
    ax.set_title(f'{label} {position}: realised loss vs VaR')
    ax.set_xlabel('Date')
    ax.set_ylabel('USD')
    ax.legend()
    ax.xaxis.set_major_locator(mdates.YearLocator(5))
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
    plt.show()

    for label, color in [('A', COLUMBIA_NAVY), ('C', COLUMBIA_DARK), ('P dire
    code = 'P' if label == 'P direct' else label
    plot_loss_vs_var(label, 'long', f'VaR_long_{code}', f'loss_long_{code}
    plot_loss_vs_var(label, 'short', f'VaR_short_{code}', f'loss_short_{c

    plot_loss_vs_var('P component MC', 'long', 'VaR_long_P_component_MC', 'lo
    plot_loss_vs_var('P component MC', 'short', 'VaR_short_P_component_MC', '
```

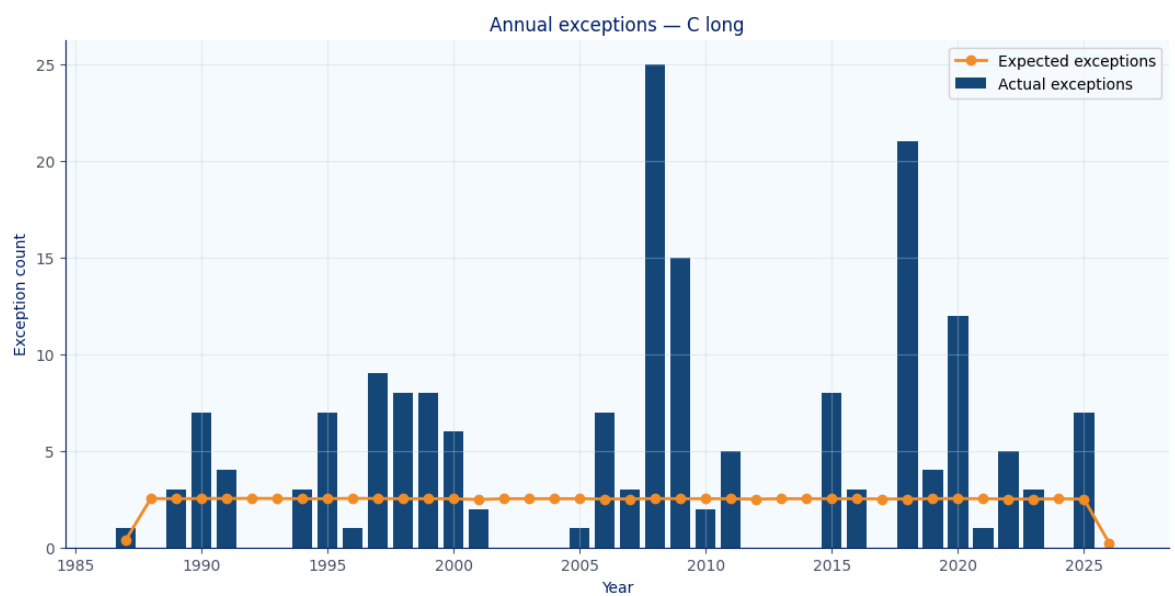
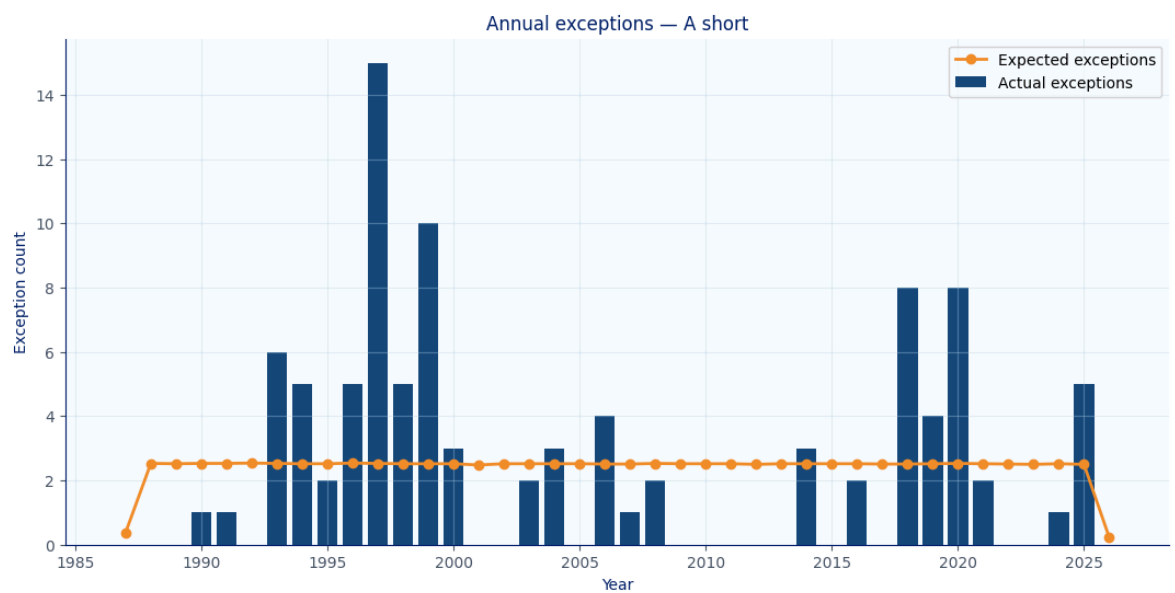
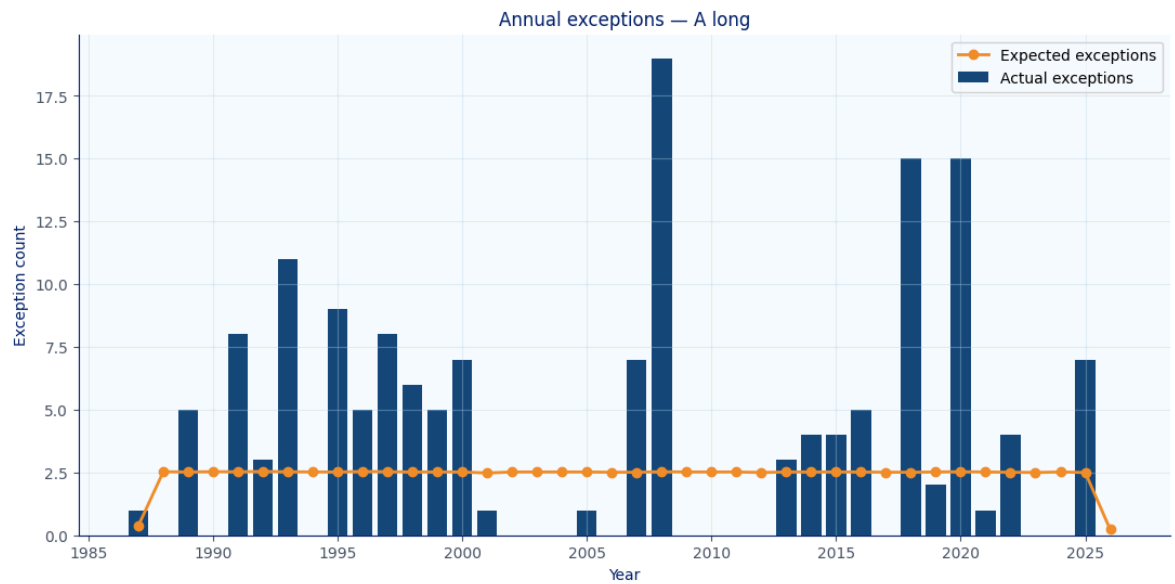


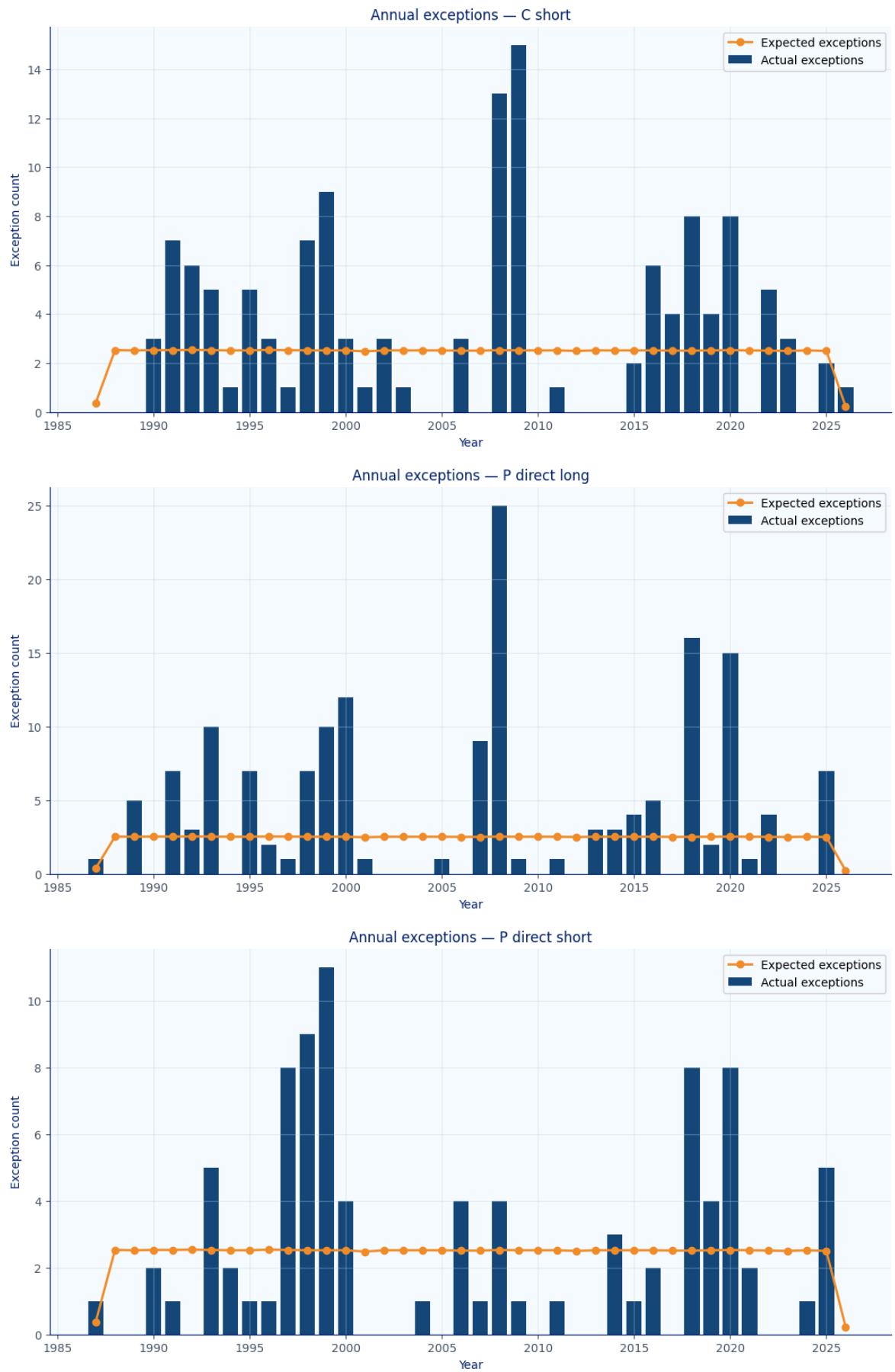


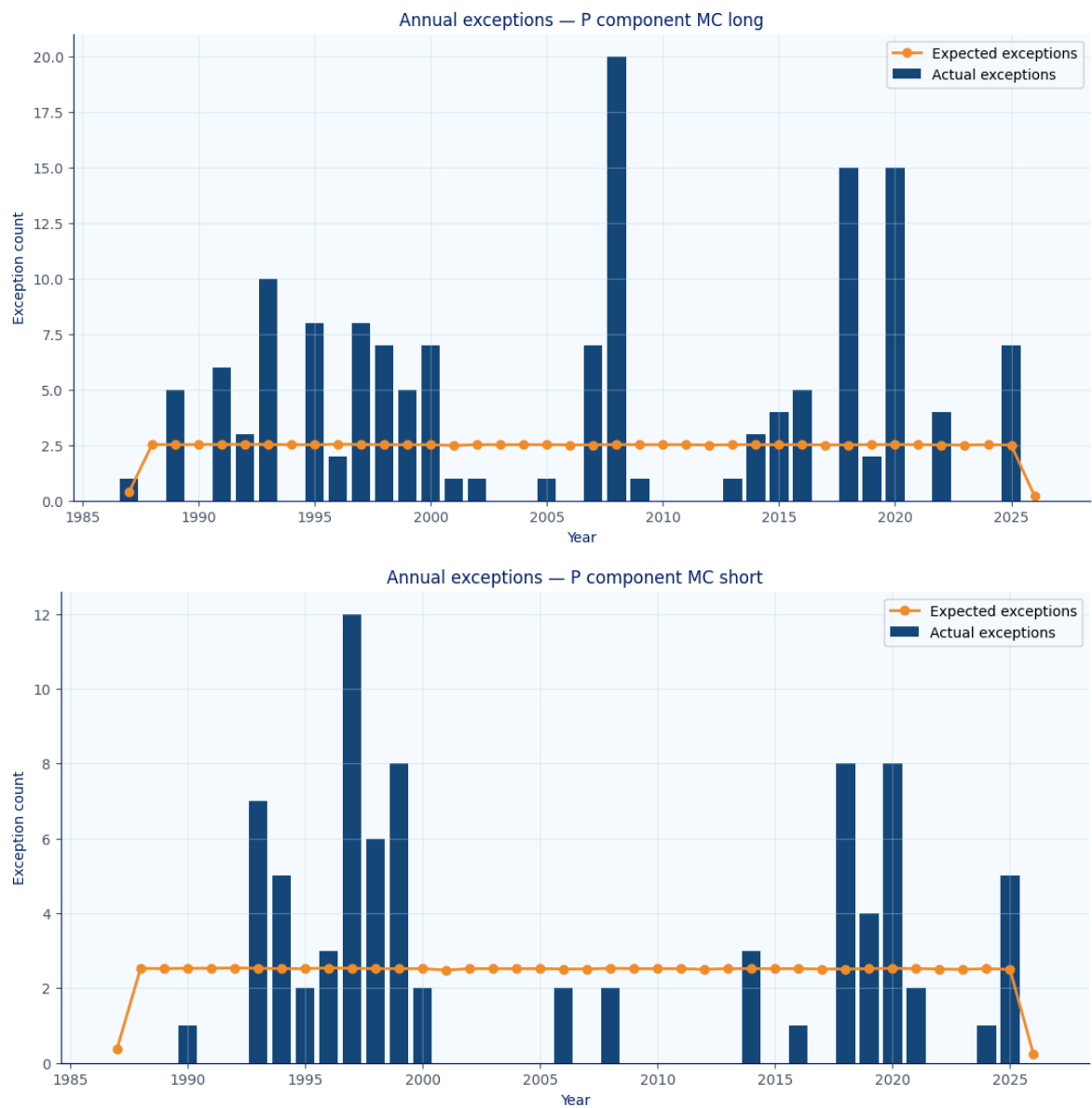


```
In [14]: def plot_annual(asset_method, position):
    sub = annual[(annual['Asset / method'] == asset_method) & (annual['Po
fig, ax = plt.subplots(figsize=(13, 6))
ax.bar(sub['year'], sub['Actual_exceptions'], color=COLUMBIA_DARK, la
ax.plot(sub['year'], sub['Expected_exceptions'], color=COLUMBIA_ORANG
ax.set_title(f'Annual exceptions - {asset_method} {position}')
ax.set_xlabel('Year')
ax.set_ylabel('Exception count')
ax.legend()
plt.show()

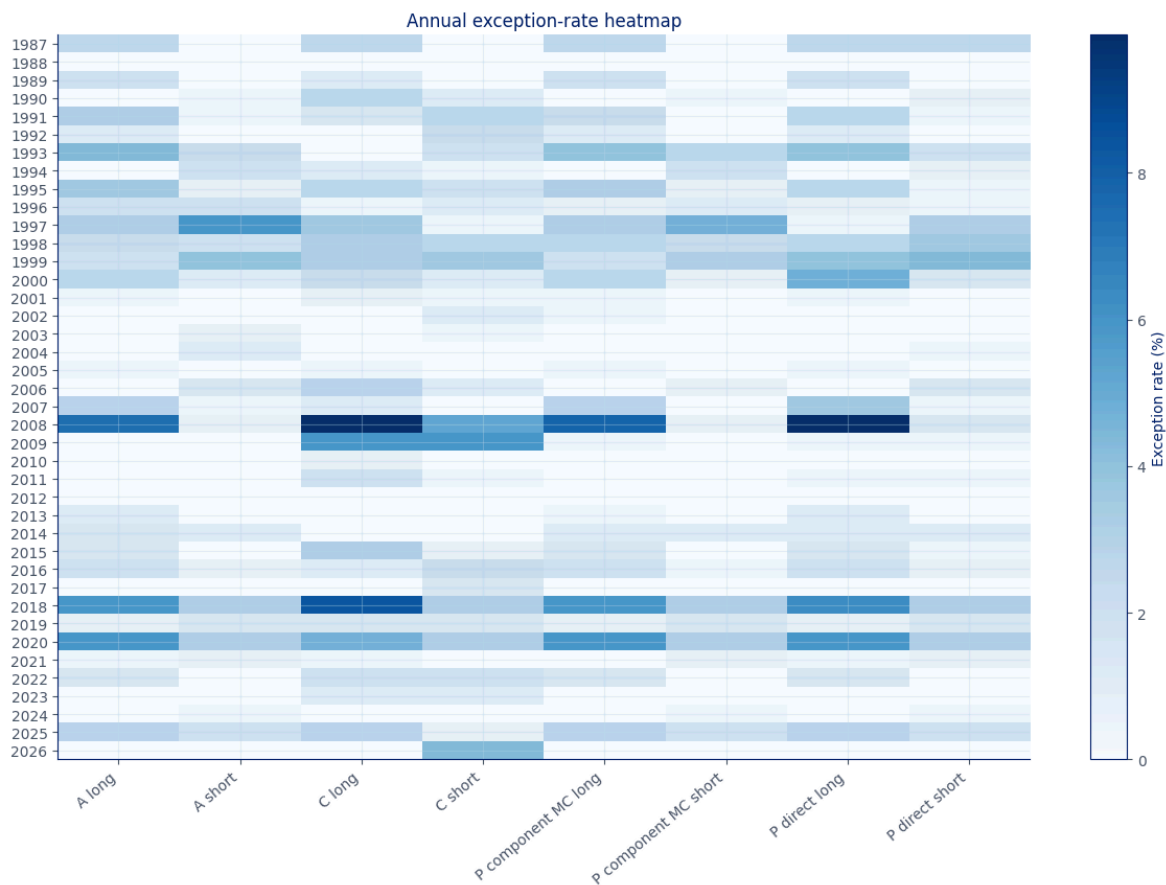
for asset_method in ['A', 'C', 'P direct', 'P component MC']:
    plot_annual(asset_method, 'long')
    plot_annual(asset_method, 'short')
```



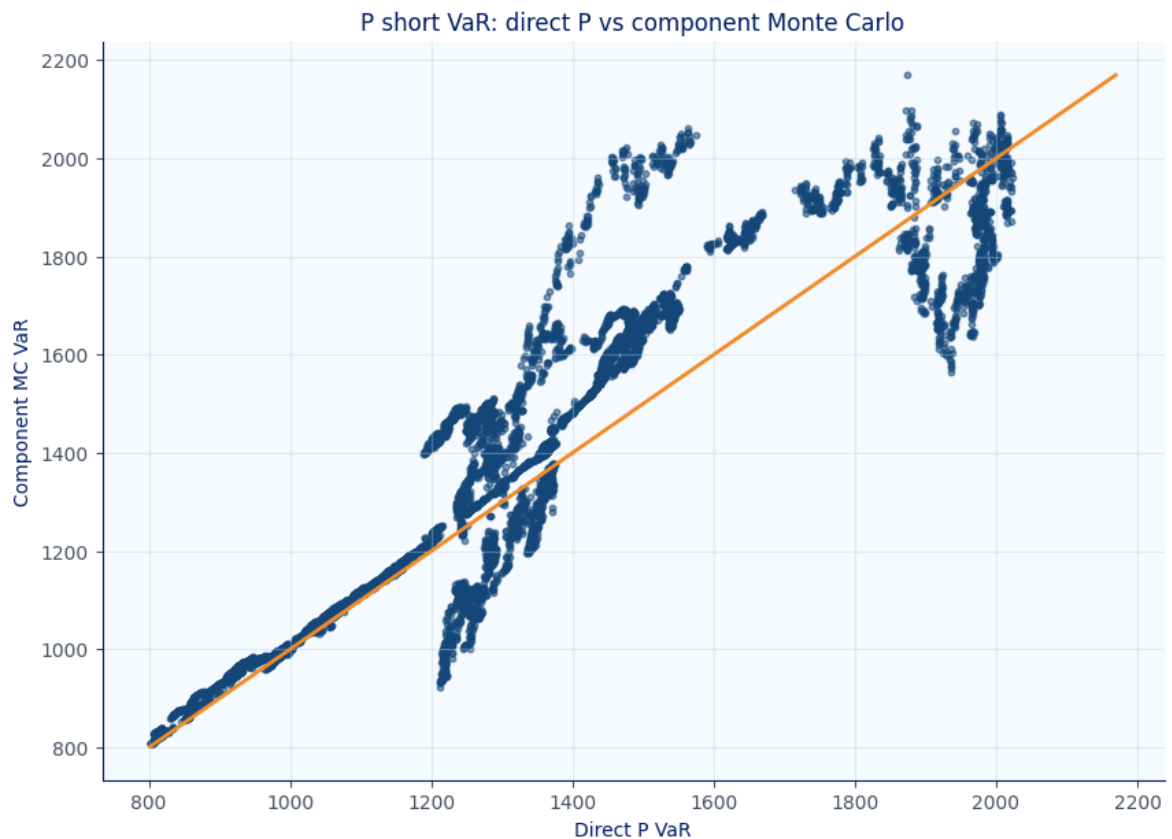




```
In [15]: heat = annual.pivot_table(index='year', columns=['Asset / method', 'Posit
fig, ax = plt.subplots(figsize=(15, 9))
im = ax.imshow(100 * heat.to_numpy(), aspect='auto', cmap='Blues')
ax.set_title('Annual exception-rate heatmap')
ax.set_yticks(np.arange(len(heat.index)))
ax.set_yticklabels(heat.index.astype(int))
ax.set_xticks(np.arange(len(heat.columns)))
ax.set_xticklabels([f'{a} {p}' for a, p in heat.columns], rotation=40, ha
cb = plt.colorbar(im)
cb.set_label('Exception rate (%)')
plt.show()
```



```
In [16]: fig, ax = plt.subplots(figsize=(10, 7))
mask = df['VaR_short_P'].notna() & df['VaR_short_P_component_MC'].notna()
ax.scatter(df.loc[mask, 'VaR_short_P'], df.loc[mask, 'VaR_short_P_compone
lo = min(df.loc[mask, 'VaR_short_P'].min(), df.loc[mask, 'VaR_short_P_com
hi = max(df.loc[mask, 'VaR_short_P'].max(), df.loc[mask, 'VaR_short_P_com
ax.plot([lo, hi], [lo, hi], color=COLUMBIA_ORANGE, linewidth=2.0)
ax.set_title('P short VaR: direct P vs component Monte Carlo')
ax.set_xlabel('Direct P VaR')
ax.set_ylabel('Component MC VaR')
plt.show()
```



7. Write-up table

This table is the main numerical result for HW10 Q1 under the corrected HW9/Q9 setup.

```
In [17]: writeup = overall.copy()
writeup['Expected exceptions'] = writeup['Expected exceptions'].round(2)
writeup['Exception rate (%)'] = (100 * writeup['Exception rate']).round(3)
writeup['Expected exception rate (%)'] = (100 * writeup['Expected exception rate']).round(3)
writeup = writeup.drop(columns=['Exception rate', 'Expected exception rate'])
display(writeup)
```

	Asset / method	Position	Backtest dates	Expected exceptions	Actual exceptions	Exception rate (%)	Expected exception rate (%)
0	A	long	9633	96.330000	156	1.619000	1.000000
1	A	short	9633	96.330000	98	1.017000	1.000000
2	C	long	9633	96.330000	181	1.879000	1.000000
3	C	short	9633	96.330000	130	1.350000	1.000000
4	P direct	long	9633	96.330000	163	1.692000	1.000000
5	P direct	short	9633	96.330000	91	0.945000	1.000000
6	P component MC	long	9633	96.330000	149	1.547000	1.000000
7	P component MC	short	9633	96.330000	82	0.851000	1.000000

8. Interpretation

The earlier SPX-based notebook was not the correct setup for this question. The correct HW10 Q1 setup uses the HW9/Q9 AAPL-CAT portfolio framework.

The expected exception count is approximately 1% of the number of usable backtest dates because the VaR confidence level is 99%.

The direct GBM backtest results show that the realised exception counts are above the expected counts for several long positions. This suggests that the 5-year GBM VaR is not fully capturing downside tail risk for AAPL, CAT, and the fixed-share portfolio during this sample.

The short-position exception counts are closer to the expected count than the long-position counts, although CAT and P still show more exceptions than expected under the direct method.

For P, the direct-P and component Monte Carlo methods produce similar but not identical results. This is expected because direct P treats the portfolio value itself as one GBM process, while the component method models AAPL and CAT separately and preserves the fixed-share portfolio structure through simulation.

9. Q2 — Exponential-equivalent backtest

Q2 repeats Q1, but the daily GBM mean, variance, and covariance estimates are replaced by the course exponential-weighting recurrence.

The exponentially weighted mean is

$$m_N = (1 - \lambda)a_N + \lambda m_{N-1}.$$

The exponentially weighted second moment is

$$r_N = (1 - \lambda)a_N^2 + \lambda r_{N-1}.$$

The variance estimate is

$$v_N = r_N - m_N^2.$$

For this assignment,

$$\lambda = 0.9989.$$

The same 99% 5-day VaR backtest is then run for long and short positions.

```
In [18]: def ewma_mean_second_var(x: pd.Series, lam: float) -> tuple[pd.Series, pd
arr = x.to_numpy(dtype=float)
m = np.full(len(arr), np.nan)
r = np.full(len(arr), np.nan)
valid = np.where(~np.isnan(arr))[0]
if len(valid) == 0:
    return pd.Series(m, index=x.index), pd.Series(r, index=x.index),
start = int(valid[0])
m[start] = arr[start]
r[start] = arr[start] ** 2
for i in range(start + 1, len(arr)):
    if np.isnan(arr[i]):
        m[i] = m[i - 1]
        r[i] = r[i - 1]
    else:
        m[i] = (1 - lam) * arr[i] + lam * m[i - 1]
        r[i] = (1 - lam) * arr[i] ** 2 + lam * r[i - 1]
v = np.maximum(r - m ** 2, 0.0)
return pd.Series(m, index=x.index), pd.Series(r, index=x.index), pd.S

def add_ewma_direct_gbm_var(data: pd.DataFrame, label: str, cfg: Config)
out = data.copy()
ret = out[f'r_{label}']
mu, second, var = ewma_mean_second_var(ret, cfg.ewma_lambda)
mu.iloc[:cfg.window_days] = np.nan
second.iloc[:cfg.window_days] = np.nan
var.iloc[:cfg.window_days] = np.nan
sig_h = np.sqrt(cfg.horizon_days * var)
mu_h = cfg.horizon_days * mu
z_left = NORMAL.inv_cdf(1 - cfg.var_level)
z_right = NORMAL.inv_cdf(cfg.var_level)
out[f'ewma_mu_{label}'] = mu
out[f'ewma_second_{label}'] = second
out[f'ewma_var_daily_{label}'] = var
out[f'ewma_vol_daily_{label}'] = np.sqrt(var)
out[f'ewma_VaR_long_{label}'] = cfg.notional * (1 - np.exp(mu_h + sig
out[f'ewma_VaR_short_{label}'] = cfg.notional * (np.exp(mu_h + sig_h
out[f'ewma_exc_long_{label}'] = out[f'ewma_VaR_long_{label}'].notna()
out[f'ewma_exc_short_{label}'] = out[f'ewma_VaR_short_{label}'].notna
return out

for label in ['A', 'C', 'P']:
    df = add_ewma_direct_gbm_var(df, label, cfg)
```



```
display(df[['Date', 'ewma_vol_daily_A', 'ewma_vol_daily_C', 'ewma_vol_dai
```

	Date	ewma_vol_daily_A	ewma_vol_daily_C	ewma_vol_daily_P	ewma_VaR_
10893	2026-02-05	0.018186	0.019194	0.017999	865.
10894	2026-02-06	0.018178	0.019312	0.017992	864.
10895	2026-02-09	0.018173	0.019314	0.017987	864.
10896	2026-02-10	0.018164	0.019303	0.017977	864.
10897	2026-02-11	0.018155	0.019342	0.017969	863.

```
In [19]: def add_ewma_component_mc_p_var(data: pd.DataFrame, cfg: Config) -> pd.DataFrame:
    out = data.copy()
    lam = cfg.ewma_lambda
    rA = out['r_A'].to_numpy(dtype=float)
    rC = out['r_C'].to_numpy(dtype=float)
    n = len(out)
    mA = np.full(n, np.nan)
    mC = np.full(n, np.nan)
    sAA = np.full(n, np.nan)
    sCC = np.full(n, np.nan)
    sAC = np.full(n, np.nan)
    valid = np.where(~np.isnan(rA) & ~np.isnan(rC))[0]
    if len(valid) > 0:
        start = int(valid[0])
        mA[start] = rA[start]
        mC[start] = rC[start]
        sAA[start] = rA[start] ** 2
        sCC[start] = rC[start] ** 2
        sAC[start] = rA[start] * rC[start]
        for i in range(start + 1, n):
            if np.isnan(rA[i]) or np.isnan(rC[i]):
                mA[i] = mA[i - 1]
                mC[i] = mC[i - 1]
                sAA[i] = sAA[i - 1]
                sCC[i] = sCC[i - 1]
                sAC[i] = sAC[i - 1]
            else:
                mA[i] = (1 - lam) * rA[i] + lam * mA[i - 1]
                mC[i] = (1 - lam) * rC[i] + lam * mC[i - 1]
                sAA[i] = (1 - lam) * rA[i] ** 2 + lam * sAA[i - 1]
                sCC[i] = (1 - lam) * rC[i] ** 2 + lam * sCC[i - 1]
                sAC[i] = (1 - lam) * rA[i] * rC[i] + lam * sAC[i - 1]
    varA = np.maximum(sAA - mA ** 2, 0.0)
    varC = np.maximum(sCC - mC ** 2, 0.0)
    covAC = sAC - mA * mC
    for arr in [mA, mC, varA, varC, covAC]:
        arr[:cfg.window_days] = np.nan
    out['ewma_mu_A_component'] = mA
```

```

out['ewma_mu_C_component'] = mC
out['ewma_var_A_component'] = varA
out['ewma_var_C_component'] = varC
out['ewma_cov_AC_component'] = covAC
rng = np.random.default_rng(cfg.mc_seed + 1)
z = rng.standard_normal((cfg.mc_paths, 2))
var_long = np.full(n, np.nan)
var_short = np.full(n, np.nan)
valid_rows = out['ewma_mu_A_component'].notna() & out['future_P'].notna()
for i in np.where(valid_rows.to_numpy())[0]:
    mu = cfg.horizon_days * np.array([out.at[i, 'ewma_mu_A_component'],
                                       out.at[i, 'ewma_mu_C_component']])
    cov = cfg.horizon_days * np.array([
        [out.at[i, 'ewma_var_A_component'], out.at[i, 'ewma_cov_AC_component']],
        [out.at[i, 'ewma_cov_AC_component'], out.at[i, 'ewma_var_C_component']]
    ])
    cov = (cov + cov.T) / 2
    vals, vecs = np.linalg.eigh(cov)
    vals = np.maximum(vals, 1e-16)
    shocks = mu + z @ (vecs @ np.diag(np.sqrt(vals))).T
    p0 = cfg.aapl_shares * out.at[i, 'A'] + cfg.cat_shares * out.at[i, 'C']
    scale = cfg.notional / p0
    pf = scale * (
        cfg.aapl_shares * out.at[i, 'A'] * np.exp(shocks[:, 0])
        + cfg.cat_shares * out.at[i, 'C'] * np.exp(shocks[:, 1])
    )
    var_long[i] = np.quantile(cfg.notional - pf, cfg.var_level)
    var_short[i] = np.quantile(pf - cfg.notional, cfg.var_level)
out['ewma_VaR_long_P_component_MC'] = var_long
out['ewma_VaR_short_P_component_MC'] = var_short
out['ewma_exc_long_P_component_MC'] = out['ewma_VaR_long_P_component_MC']
out['ewma_exc_short_P_component_MC'] = out['ewma_VaR_short_P_component_MC']
return out

df = add_ewma_component_mc_p_var(df, cfg)
display(df[['Date', 'ewma_VaR_long_P', 'ewma_VaR_long_P_component_MC', 'ewma_VaR_sh

```

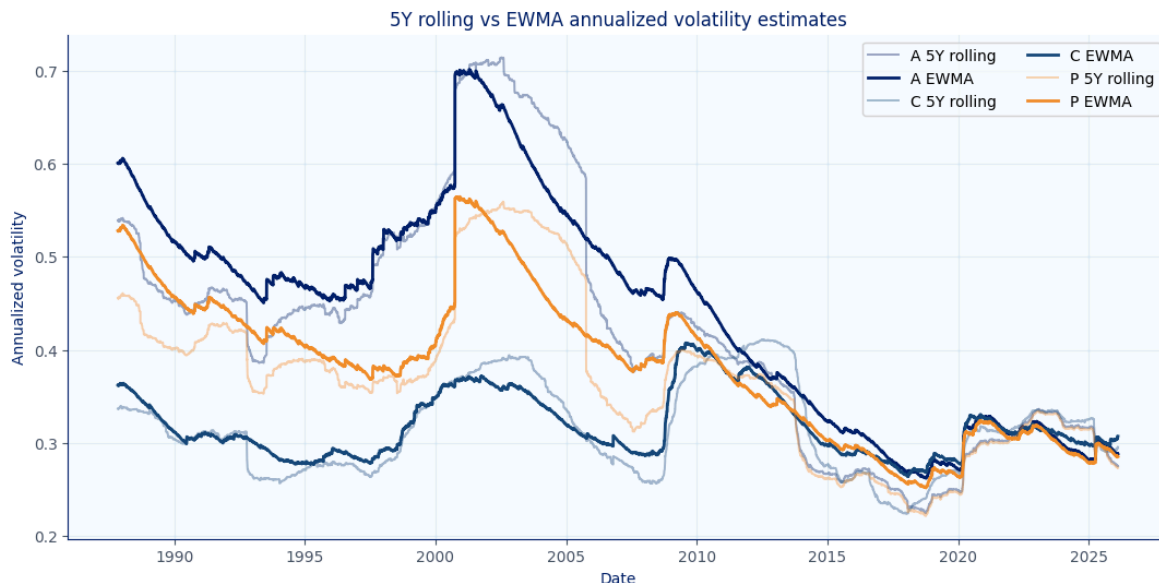
	Date	ewma_VaR_long_P	ewma_VaR_long_P_component_MC	ewma_VaR_sh
10893	2026-02-05	856.086809	NaN	1,027.00
10894	2026-02-06	855.301290	NaN	1,027.07
10895	2026-02-09	855.719260	NaN	1,026.00
10896	2026-02-10	855.484251	NaN	1,025.22
10897	2026-02-11	854.748464	NaN	1,025.12

```

In [20]: fig, ax = plt.subplots(figsize=(13, 6))
for label, color in [('A', COLUMBIA_NAVY), ('C', COLUMBIA_DARK), ('P', CO
ax.plot(df['Date'], df[f'vol_daily_{label}'] * np.sqrt(cfg.trading_da
ax.plot(df['Date'], df[f'ewma_vol_daily_{label}'] * np.sqrt(cfg.tradi
ax.set_title('5Y rolling vs EWMA annualized volatility estimates')

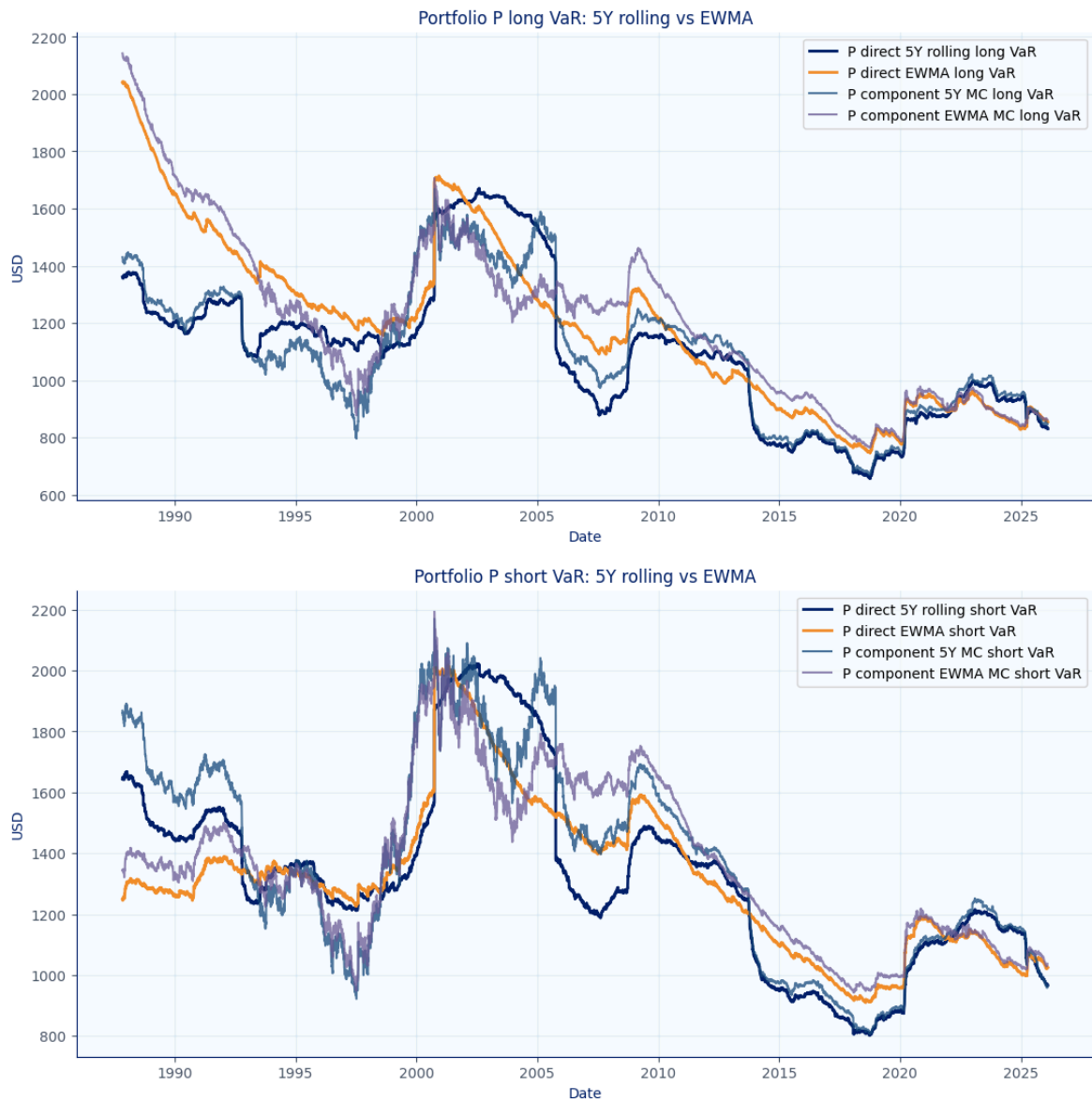
```

```
ax.set_xlabel('Date')
ax.set_ylabel('Annualized volatility')
ax.legend(ncol=2)
ax.xaxis.set_major_locator(mdates.YearLocator(5))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.show()
```



```
In [21]: fig, ax = plt.subplots(figsize=(13, 6))
ax.plot(df['Date'], df['VaR_long_P'], color=COLUMBIA_NAVY, linewidth=2.0,
ax.plot(df['Date'], df['ewma_VaR_long_P'], color=COLUMBIA_ORANGE, linewid
ax.plot(df['Date'], df['VaR_long_P_component_MC'], color=COLUMBIA_DARK, a
ax.plot(df['Date'], df['ewma_VaR_long_P_component_MC'], color=COLUMBIA_PU
ax.set_title('Portfolio P long VaR: 5Y rolling vs EWMA')
ax.set_xlabel('Date')
ax.set_ylabel('USD')
ax.legend()
ax.xaxis.set_major_locator(mdates.YearLocator(5))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.show()

fig, ax = plt.subplots(figsize=(13, 6))
ax.plot(df['Date'], df['VaR_short_P'], color=COLUMBIA_NAVY, linewidth=2.0
ax.plot(df['Date'], df['ewma_VaR_short_P'], color=COLUMBIA_ORANGE, linewi
ax.plot(df['Date'], df['VaR_short_P_component_MC'], color=COLUMBIA_DARK,
ax.plot(df['Date'], df['ewma_VaR_short_P_component_MC'], color=COLUMBIA_P
ax.set_title('Portfolio P short VaR: 5Y rolling vs EWMA')
ax.set_xlabel('Date')
ax.set_ylabel('USD')
ax.legend()
ax.xaxis.set_major_locator(mdates.YearLocator(5))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.show()
```



```
In [22]: def ewma_overall_summary(data: pd.DataFrame, cfg: Config) -> pd.DataFrame
        specs = [
            ('A', 'long', 'ewma_VaR_long_A', 'loss_long_A', 'ewma_exc_long_A',
             'A', 'short', 'ewma_VaR_short_A', 'loss_short_A', 'ewma_exc_shor
            ('C', 'long', 'ewma_VaR_long_C', 'loss_long_C', 'ewma_exc_long_C',
            ('C', 'short', 'ewma_VaR_short_C', 'loss_short_C', 'ewma_exc_shor
            ('P direct', 'long', 'ewma_VaR_long_P', 'loss_long_P', 'ewma_exc_
            ('P direct', 'short', 'ewma_VaR_short_P', 'loss_short_P', 'ewma_e
            ('P component MC', 'long', 'ewma_VaR_long_P_component_MC', 'loss_
            ('P component MC', 'short', 'ewma_VaR_short_P_component_MC', 'los
        ]
        rows = []
        for asset, position, var_col, loss_col, exc_col in specs:
            usable = data[var_col].notna() & data[loss_col].notna()
            n = int(usable.sum())
            actual = int(data.loc[usable, exc_col].sum())
            rows.append({
                'Estimator': f'EWMA lambda={cfg.ewma_lambda}',
                'Asset / method': asset,
                'Position': position,
                'Backtest dates': n,
                'Expected exceptions': (1 - cfg.var_level) * n,
                'Actual exceptions': actual,
                'Exception rate': actual / n if n else np.nan,
```

```

        'Expected exception rate': 1 - cfg.var_level,
    })
    return pd.DataFrame(rows)

q1_overall = overall.copy()
q1_overall.insert(0, 'Estimator', '5Y rolling')
q2_overall = ewma_overall_summary(df, cfg)
combined_overall = pd.concat([q1_overall, q2_overall], ignore_index=True)

display(Markdown('### Q1 and Q2 combined exception summary'))
display(combined_overall)

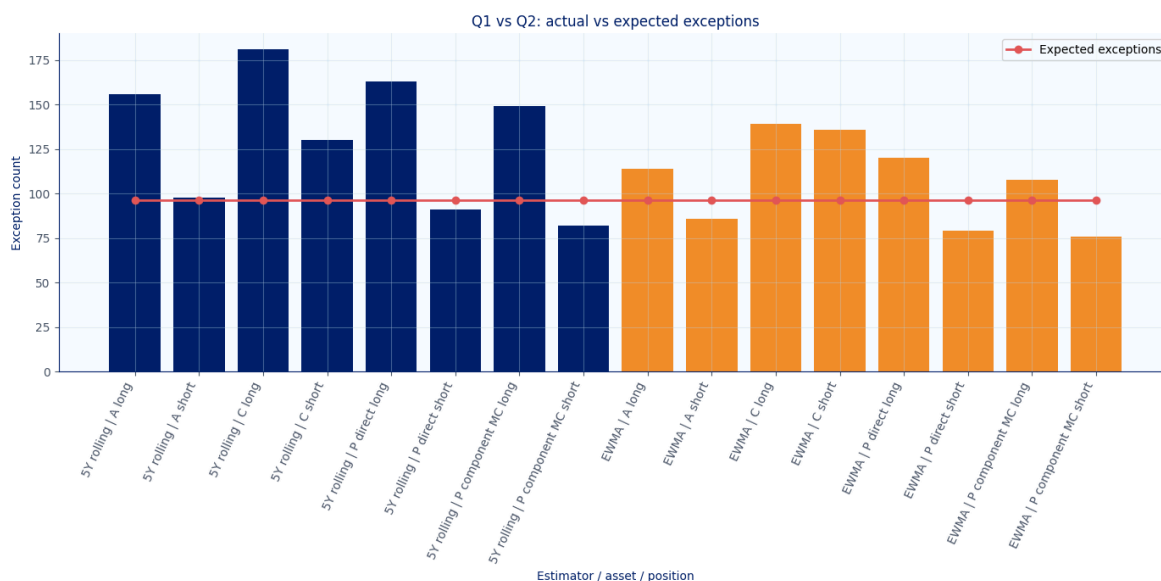
```

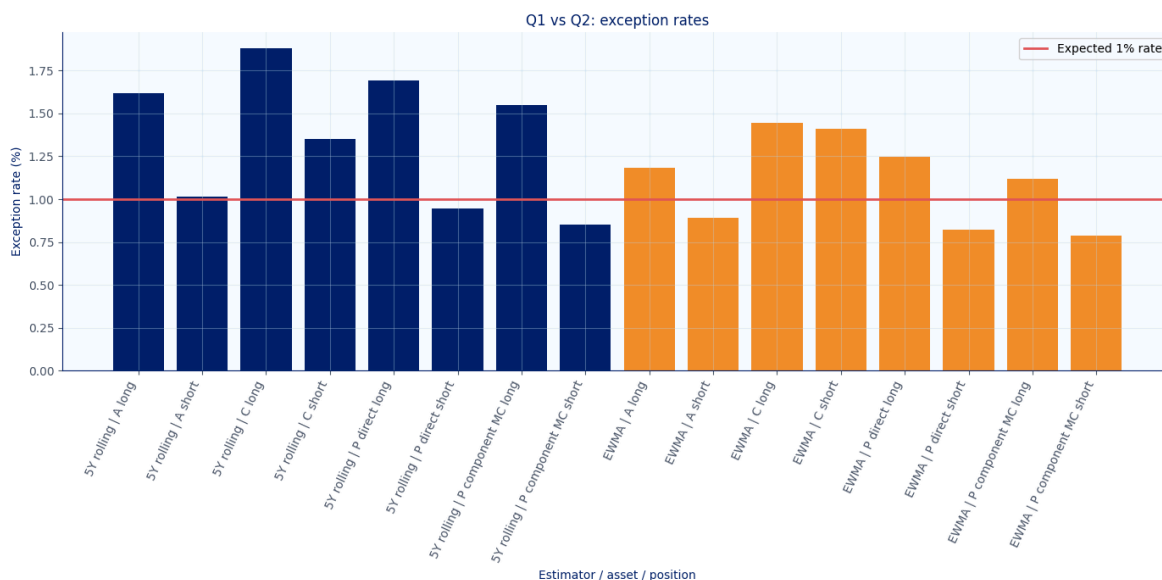
Q1 and Q2 combined exception summary

	Estimator	Asset / method	Position	Backtest dates	Expected exceptions	Actual exceptions	Exception rate
0	5Y rolling	A	long	9633	96.330000	156	0.016194
1	5Y rolling	A	short	9633	96.330000	98	0.010173
2	5Y rolling	C	long	9633	96.330000	181	0.018790
3	5Y rolling	C	short	9633	96.330000	130	0.013495
4	5Y rolling	P direct	long	9633	96.330000	163	0.016927
5	5Y rolling	P direct	short	9633	96.330000	91	0.009447
6	5Y rolling	P component MC	long	9633	96.330000	149	0.015468
7	5Y rolling	P component MC	short	9633	96.330000	82	0.008512
8	EWMA lambda=0.9989	A	long	9633	96.330000	114	0.011834
9	EWMA lambda=0.9989	A	short	9633	96.330000	86	0.008928
10	EWMA lambda=0.9989	C	long	9633	96.330000	139	0.014430
11	EWMA lambda=0.9989	C	short	9633	96.330000	136	0.014118
12	EWMA lambda=0.9989	P direct	long	9633	96.330000	120	0.012457
13	EWMA lambda=0.9989	P direct	short	9633	96.330000	79	0.008207
14	EWMA lambda=0.9989	P component MC	long	9633	96.330000	108	0.011217
15	EWMA lambda=0.9989	P component MC	short	9633	96.330000	76	0.007890

```
In [23]: fig, ax = plt.subplots(figsize=(14, 7))
plot_data = combined_overall.copy()
plot_data['label'] = plot_data['Estimator'].str.replace(f' lambda={cfg.ew
x = np.arange(len(plot_data))
ax.bar(x, plot_data['Actual exceptions'], color=[COLUMBIA_NAVY if e == '5
ax.plot(x, plot_data['Expected exceptions'], color=COLUMBIA_RED, marker='
ax.set_title('Q1 vs Q2: actual vs expected exceptions')
ax.set_xlabel('Estimator / asset / position')
ax.set_ylabel('Exception count')
ax.set_xticks(x)
ax.set_xticklabels(plot_data['label'], rotation=65, ha='right')
ax.legend()
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(14, 7))
ax.bar(x, 100 * plot_data['Exception rate'], color=[COLUMBIA_NAVY if e ==
ax.axhline(100 * (1 - cfg.var_level), color=COLUMBIA_RED, linewidth=2.0,
ax.set_title('Q1 vs Q2: exception rates')
ax.set_xlabel('Estimator / asset / position')
ax.set_ylabel('Exception rate (%)')
ax.set_xticks(x)
ax.set_xticklabels(plot_data['label'], rotation=65, ha='right')
ax.legend()
plt.tight_layout()
plt.show()
```

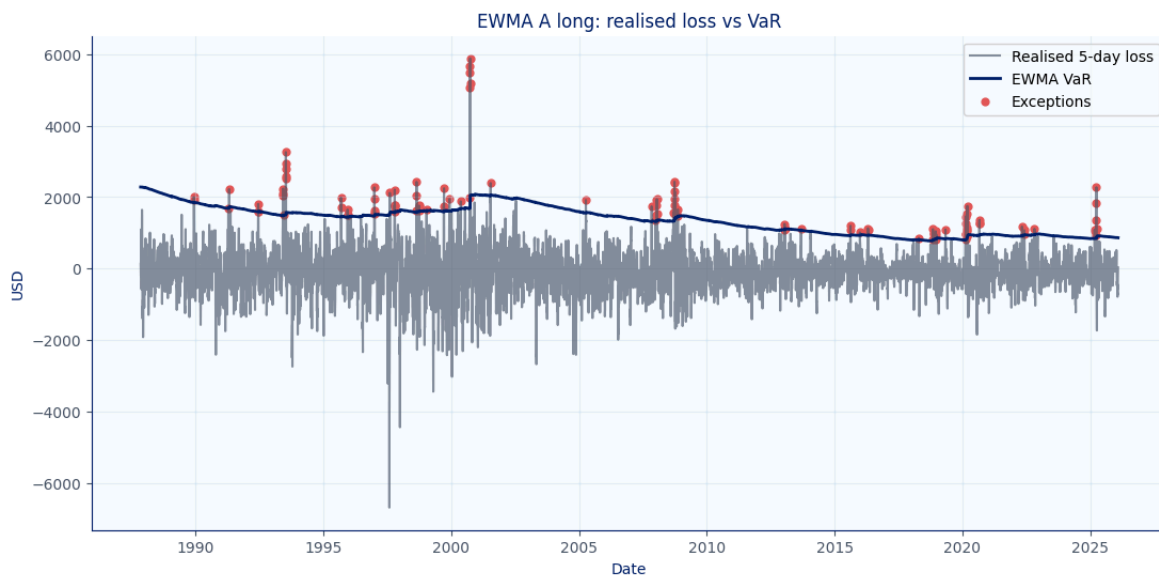


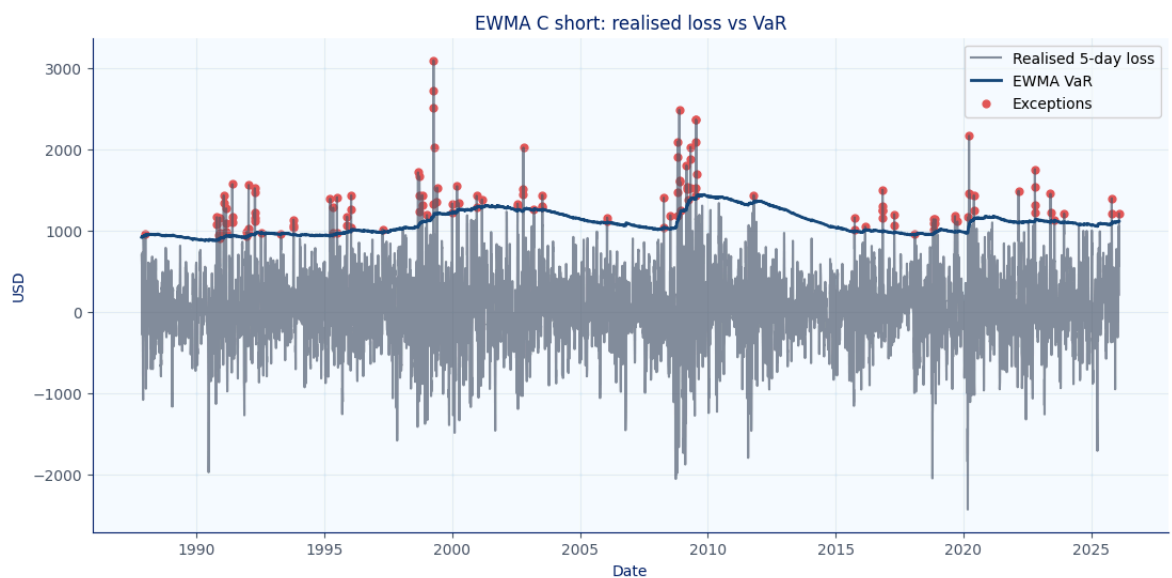
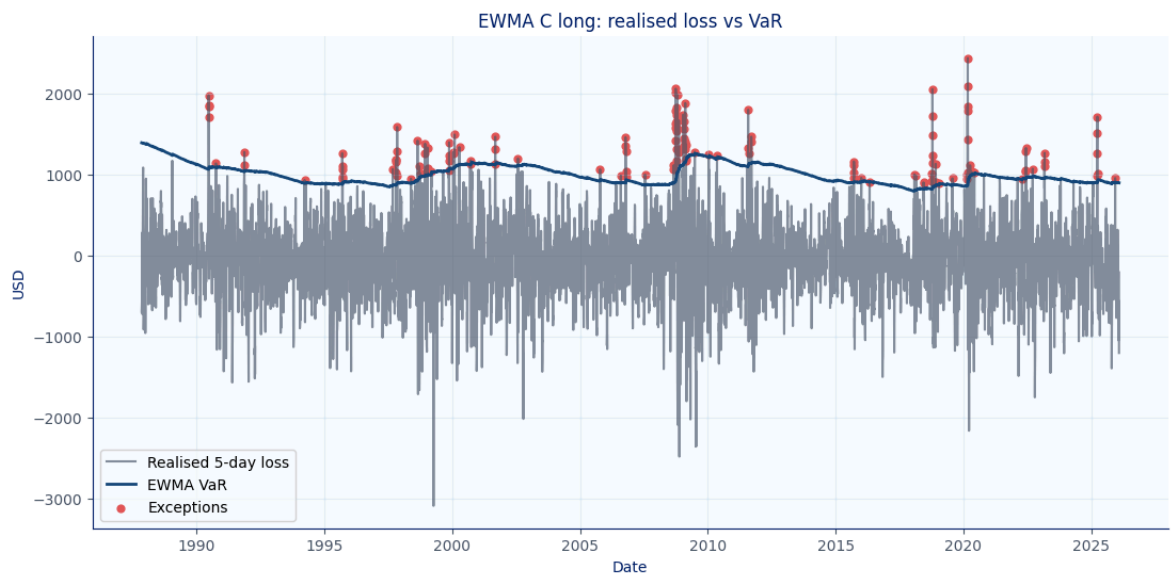
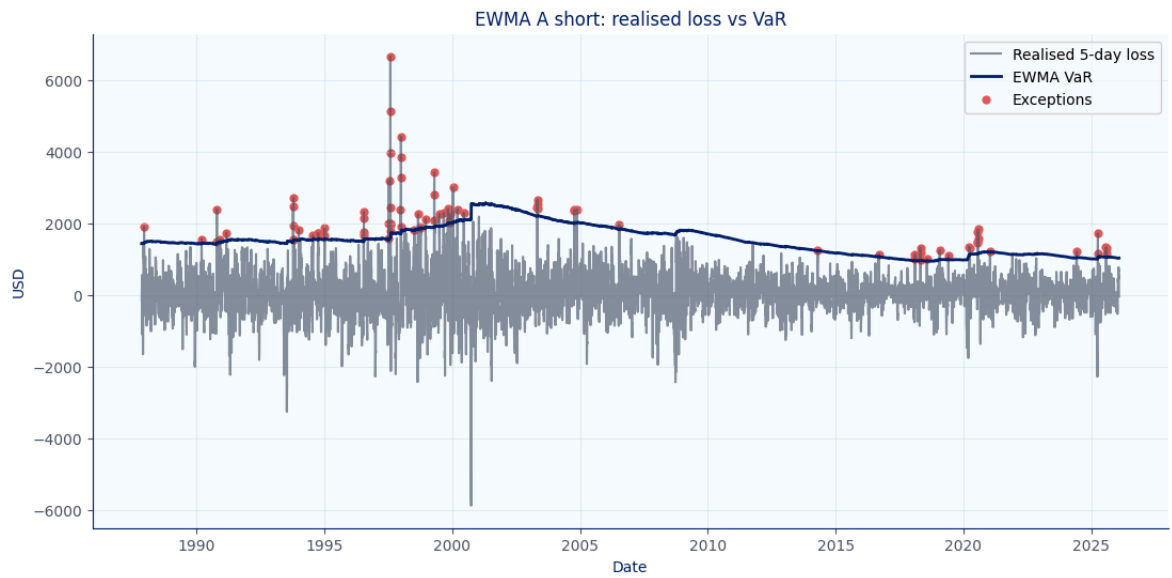


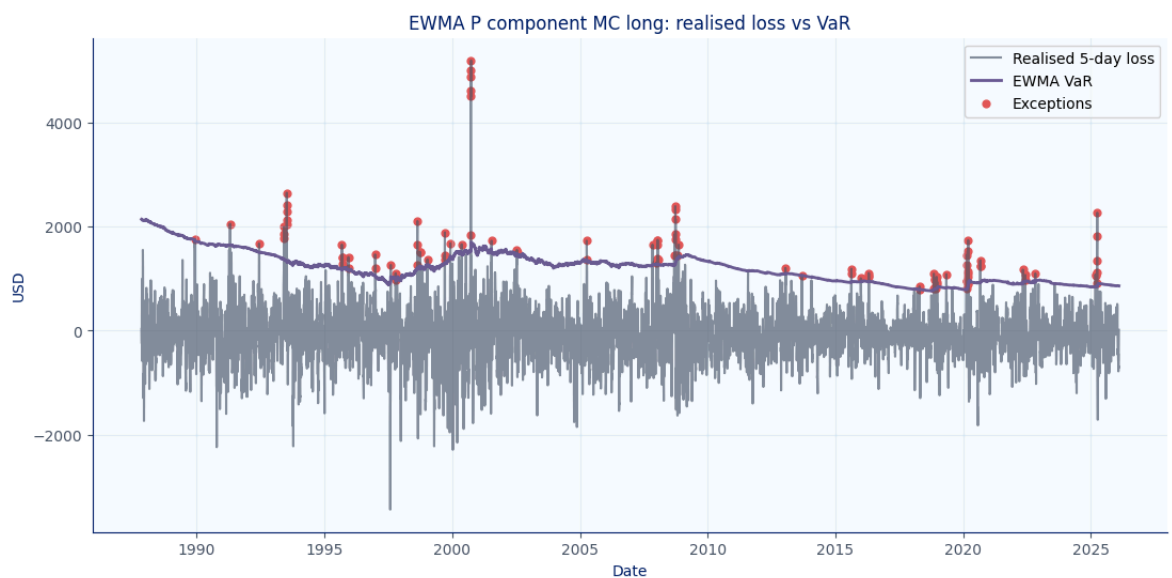
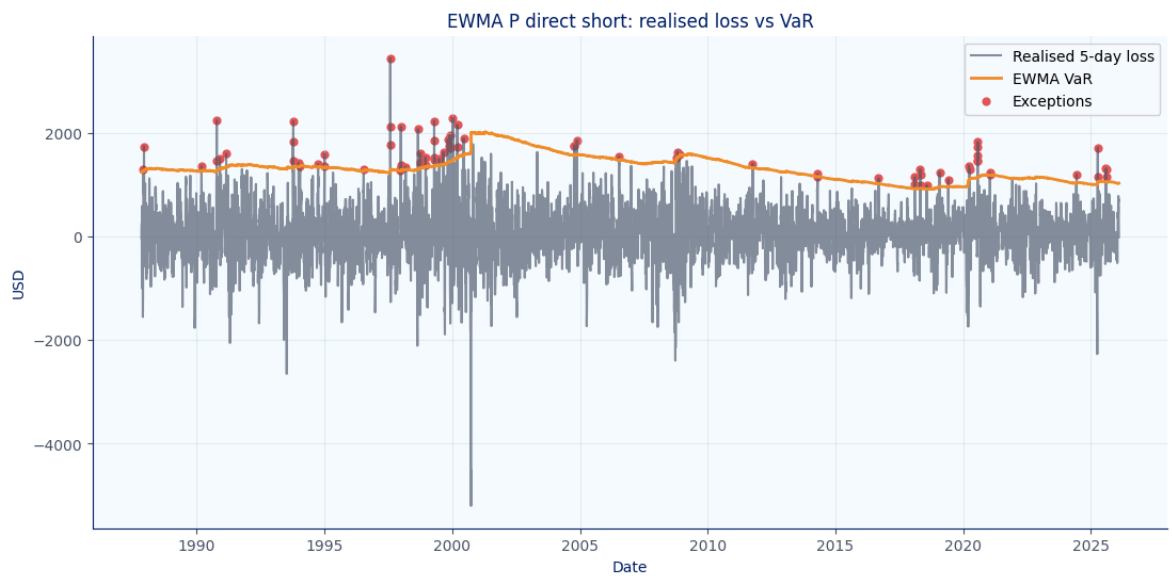
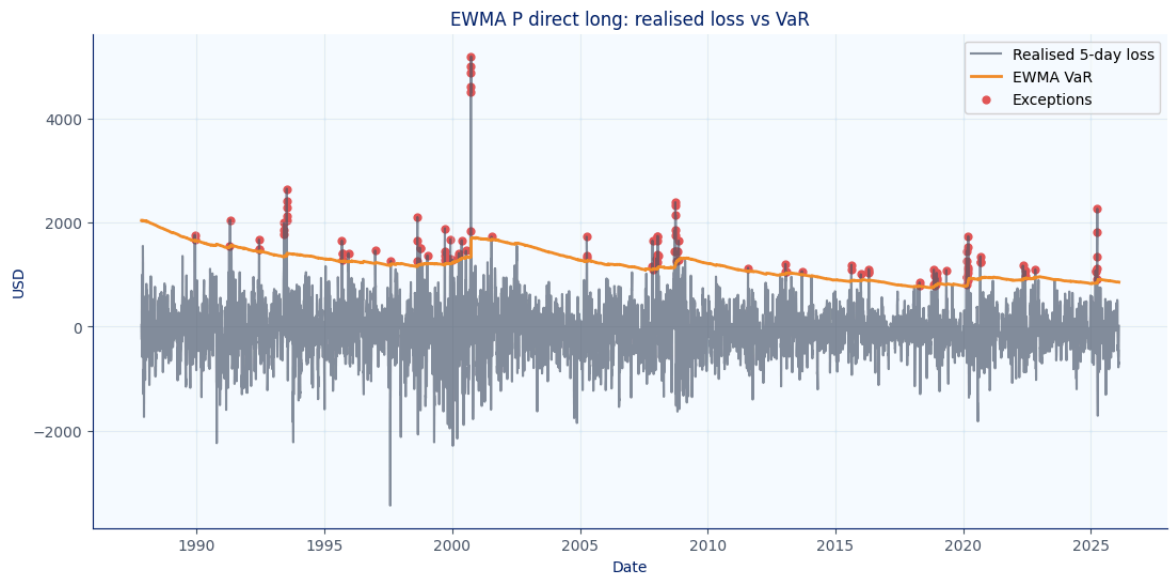
```
In [24]: def plot_ewma_loss_vs_var(label, position, var_col, loss_col, exc_col, co
usable = df[var_col].notna() & df[loss_col].notna()
sub = df.loc[usable]
fig, ax = plt.subplots(figsize=(13, 6))
ax.plot(sub['Date'], sub[loss_col], color=COLUMBIA_GRAY, alpha=0.65,
ax.plot(sub['Date'], sub[var_col], color=color, linewidth=2.0, label=
exc = sub[sub[exc_col]]
ax.scatter(exc['Date'], exc[loss_col], color=COLUMBIA_RED, s=24, labe
ax.set_title(f'EWMA {label} {position}: realised loss vs VaR')
ax.set_xlabel('Date')
ax.set_ylabel('USD')
ax.legend()
ax.xaxis.set_major_locator(mdates.YearLocator(5))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.show()

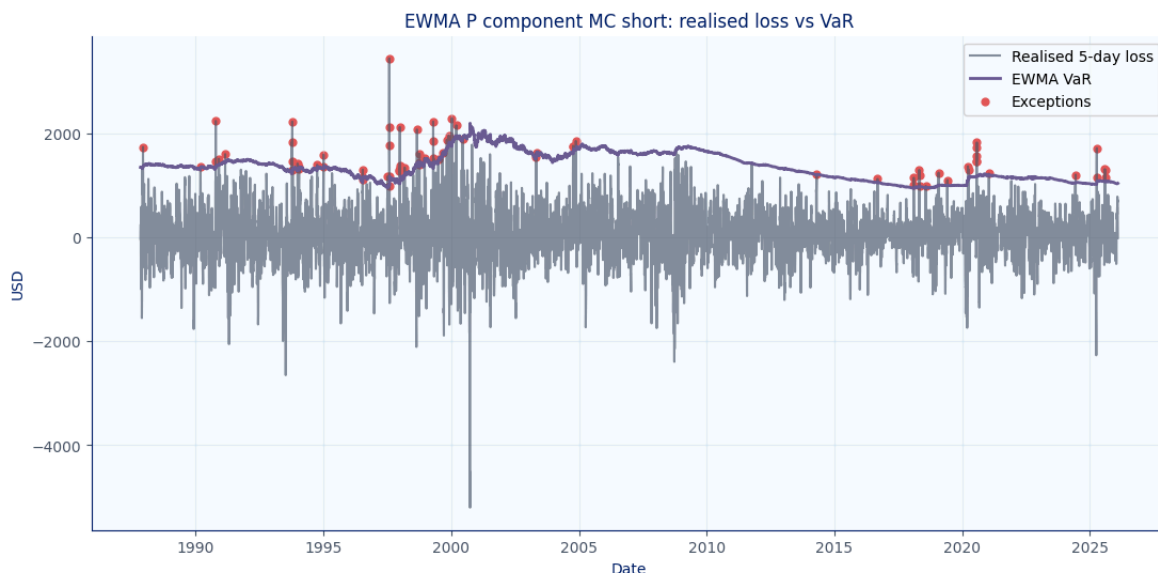
for label, color in [('A', COLUMBIA_NAVY), ('C', COLUMBIA_DARK), ('P dire
code = 'P' if label == 'P direct' else label
plot_ewma_loss_vs_var(label, 'long', f'ewma_VaR_long_{code}', f'loss_
plot_ewma_loss_vs_var(label, 'short', f'ewma_VaR_short_{code}', f'los

plot_ewma_loss_vs_var('P component MC', 'long', 'ewma_VaR_long_P_componen
plot_ewma_loss_vs_var('P component MC', 'short', 'ewma_VaR_short_P_compon
```









10. 97.5% VaR sensitivity check

This final section repeats the same backtest at a lower VaR confidence level, $q = 97.5\%$, while keeping the same data, same 5-day horizon, same USD 10,000 exposure, same portfolio definition, and same 5-year burn-in.

At $q = 97.5\%$, the expected exception rate is 2.5% rather than 1% , so the expected number of exceptions is

$$(1 - 0.975)N = 0.025N.$$

With 9,633 usable dates, this benchmark is approximately

$$0.025 \times 9633 = 240.825.$$

```
In [25]: VAR_LEVEL_975 = 0.975
SUFFIX_975 = '975'

def add_direct_var_for_level(data: pd.DataFrame, label: str, level: float)
    out = data.copy()
    mu = out[f'mu_{label}']
    var = out[f'var_daily_{label}']
    mu_h = cfg.horizon_days * mu
    sig_h = np.sqrt(cfg.horizon_days * var)
    z_left = NORMAL.inv_cdf(1 - level)
    z_right = NORMAL.inv_cdf(level)
    out[f'VaR{suffix}_long_{label}'] = cfg.notional * (1 - np.exp(mu_h +
    out[f'VaR{suffix}_short_{label}'] = cfg.notional * (np.exp(mu_h + sig
    out[f'exc{suffix}_long_{label}'] = out[f'VaR{suffix}_long_{label}'].n
    out[f'exc{suffix}_short_{label}'] = out[f'VaR{suffix}_short_{label}']
    return out

for label in ['A', 'C', 'P']:
    df = add_direct_var_for_level(df, label, VAR_LEVEL_975, SUFFIX_975, c
```

```
In [26]: def add_component_mc_var_for_level(
    data: pd.DataFrame,
    level: float,
```

```

    cfg: Config,
    estimator_prefix: str,
    output_prefix: str,
    exception_prefix: str,
    seed_offset: int,
) -> pd.DataFrame:
    out = data.copy()
    rng = np.random.default_rng(cfg.mc_seed + seed_offset)
    z = rng.standard_normal((cfg.mc_paths, 2))
    var_long = np.full(len(out), np.nan)
    var_short = np.full(len(out), np.nan)

    if estimator_prefix == 'rolling':
        mu_a_col = 'mu_A_component'
        mu_c_col = 'mu_C_component'
        var_a_col = 'var_A_component'
        var_c_col = 'var_C_component'
        cov_col = 'cov_AC_component'
    else:
        mu_a_col = 'ewma_mu_A_component'
        mu_c_col = 'ewma_mu_C_component'
        var_a_col = 'ewma_var_A_component'
        var_c_col = 'ewma_var_C_component'
        cov_col = 'ewma_cov_AC_component'

    valid = out[mu_a_col].notna() & out['future_P'].notna()

    for i in np.where(valid.to_numpy())[0]:
        mu = cfg.horizon_days * np.array([out.at[i, mu_a_col], out.at[i,
        cov = cfg.horizon_days * np.array([
            [out.at[i, var_a_col], out.at[i, cov_col]],
            [out.at[i, cov_col], out.at[i, var_c_col]],
        ])
        cov = (cov + cov.T) / 2
        vals, vecs = np.linalg.eigh(cov)
        vals = np.maximum(vals, 1e-16)
        shocks = mu + z @ (vecs @ np.diag(np.sqrt(vals))).T
        p0 = cfg.aapl_shares * out.at[i, 'A'] + cfg.cat_shares * out.at[i
        scale = cfg.notional / p0
        pf = scale * (
            cfg.aapl_shares * out.at[i, 'A'] * np.exp(shocks[:, 0])
            + cfg.cat_shares * out.at[i, 'C'] * np.exp(shocks[:, 1])
        )
        var_long[i] = np.quantile(cfg.notional - pf, level)
        var_short[i] = np.quantile(pf - cfg.notional, level)

    out[f'{output_prefix}_long_P_component_MC'] = var_long
    out[f'{output_prefix}_short_P_component_MC'] = var_short
    out[f'{exception_prefix}_long_P_component_MC'] = out[f'{output_prefix
    out[f'{exception_prefix}_short_P_component_MC'] = out[f'{output_prefi
    return out

df = add_component_mc_var_for_level(df, VAR_LEVEL_975, cfg, 'rolling', 'V
df = add_component_mc_var_for_level(df, VAR_LEVEL_975, cfg, 'ewma', 'ewma

```

```

In [27]: def add_ewma_direct_var_for_level(data: pd.DataFrame, label: str, level:
    out = data.copy()
    mu = out[f'ewma_mu_{label}']
    var = out[f'ewma_var_daily_{label}']
    mu_h = cfg.horizon_days * mu

```

```

sig_h = np.sqrt(cfg.horizon_days * var)
z_left = NORMAL.inv_cdf(1 - level)
z_right = NORMAL.inv_cdf(level)
out[f'ewma_VaR{suffix}_long_{label}'] = cfg.notional * (1 - np.exp(mu
out[f'ewma_VaR{suffix}_short_{label}'] = cfg.notional * (np.exp(mu_h
out[f'ewma_exc{suffix}_long_{label}'] = out[f'ewma_VaR{suffix}_long_{
out[f'ewma_exc{suffix}_short_{label}'] = out[f'ewma_VaR{suffix}_short
return out

for label in ['A', 'C', 'P']:
    df = add_ewma_direct_var_for_level(df, label, VAR_LEVEL_975, SUFFIX_9

```

```

In [28]: def summary_for_var_level(data: pd.DataFrame, level: float, suffix: str,
    specs = [
        ('5Y rolling', 'A', 'long', f'VaR{suffix}_long_A', 'loss_long_A',
        ('5Y rolling', 'A', 'short', f'VaR{suffix}_short_A', 'loss_short_
        ('5Y rolling', 'C', 'long', f'VaR{suffix}_long_C', 'loss_long_C',
        ('5Y rolling', 'C', 'short', f'VaR{suffix}_short_C', 'loss_short_
        ('5Y rolling', 'P direct', 'long', f'VaR{suffix}_long_P', 'loss_l
        ('5Y rolling', 'P direct', 'short', f'VaR{suffix}_short_P', 'loss
        ('5Y rolling', 'P component MC', 'long', f'VaR{suffix}_long_P_com
        ('5Y rolling', 'P component MC', 'short', f'VaR{suffix}_short_P_c
        (f'EWMA lambda={cfg.ewma_lambda}', 'A', 'long', f'ewma_VaR{suffix
        (f'EWMA lambda={cfg.ewma_lambda}', 'A', 'short', f'ewma_VaR{suffi
        (f'EWMA lambda={cfg.ewma_lambda}', 'C', 'long', f'ewma_VaR{suffix
        (f'EWMA lambda={cfg.ewma_lambda}', 'C', 'short', f'ewma_VaR{suffi
        (f'EWMA lambda={cfg.ewma_lambda}', 'P direct', 'long', f'ewma_VaR
        (f'EWMA lambda={cfg.ewma_lambda}', 'P direct', 'short', f'ewma_Va
        (f'EWMA lambda={cfg.ewma_lambda}', 'P component MC', 'long', f'ew
        (f'EWMA lambda={cfg.ewma_lambda}', 'P component MC', 'short', f'e
    ]
    rows = []
    for estimator, asset, position, var_col, loss_col, exc_col in specs:
        usable = data[var_col].notna() & data[loss_col].notna()
        n = int(usable.sum())
        actual = int(data.loc[usable, exc_col].sum())
        rows.append({
            'VaR level': level,
            'Estimator': estimator,
            'Asset / method': asset,
            'Position': position,
            'Backtest dates': n,
            'Expected exceptions': (1 - level) * n,
            'Actual exceptions': actual,
            'Exception rate': actual / n if n else np.nan,
            'Expected exception rate': 1 - level,
        })
    return pd.DataFrame(rows)

summary_975 = summary_for_var_level(df, VAR_LEVEL_975, SUFFIX_975, cfg)
display(Markdown('### 97.5% VaR exception summary'))
display(summary_975)

```

97.5% VaR exception summary

	VaR level	Estimator	Asset / method	Position	Backtest dates	Expected exceptions	Actual exceptions
0	0.975000	5Y rolling	A	long	9633	240.825000	238
1	0.975000	5Y rolling	A	short	9633	240.825000	180
2	0.975000	5Y rolling	C	long	9633	240.825000	306
3	0.975000	5Y rolling	C	short	9633	240.825000	273
4	0.975000	5Y rolling	P direct	long	9633	240.825000	250
5	0.975000	5Y rolling	P direct	short	9633	240.825000	187
6	0.975000	5Y rolling	P component MC	long	9633	240.825000	238
7	0.975000	5Y rolling	P component MC	short	9633	240.825000	169
8	0.975000	EWMA lambda=0.9989	A	long	9633	240.825000	184
9	0.975000	EWMA lambda=0.9989	A	short	9633	240.825000	187
10	0.975000	EWMA lambda=0.9989	C	long	9633	240.825000	268
11	0.975000	EWMA lambda=0.9989	C	short	9633	240.825000	276
12	0.975000	EWMA lambda=0.9989	P direct	long	9633	240.825000	194
13	0.975000	EWMA lambda=0.9989	P direct	short	9633	240.825000	183
14	0.975000	EWMA lambda=0.9989	P component MC	long	9633	240.825000	185
15	0.975000	EWMA lambda=0.9989	P component MC	short	9633	240.825000	165

```
In [29]: summary_99_for_compare = combined_overall.copy()
summary_99_for_compare.insert(0, 'VaR level', cfg.var_level)
summary_975_for_compare = summary_975.copy()
level_compare = pd.concat([summary_99_for_compare, summary_975_for_compare])

compact_compare = level_compare.copy()
compact_compare['Expected exceptions'] = compact_compare['Expected exceptions']
compact_compare['Exception rate (%)'] = (100 * compact_compare['Exception rate (%)'])
compact_compare['Expected exception rate (%)'] = (100 * compact_compare['Expected exception rate (%)'])
compact_compare = compact_compare.drop(columns=['Exception rate', 'Expected exception rate'])

display(Markdown('### 99% vs 97.5% comparison table'))
display(compact_compare)
```

99% vs 97.5% comparison table

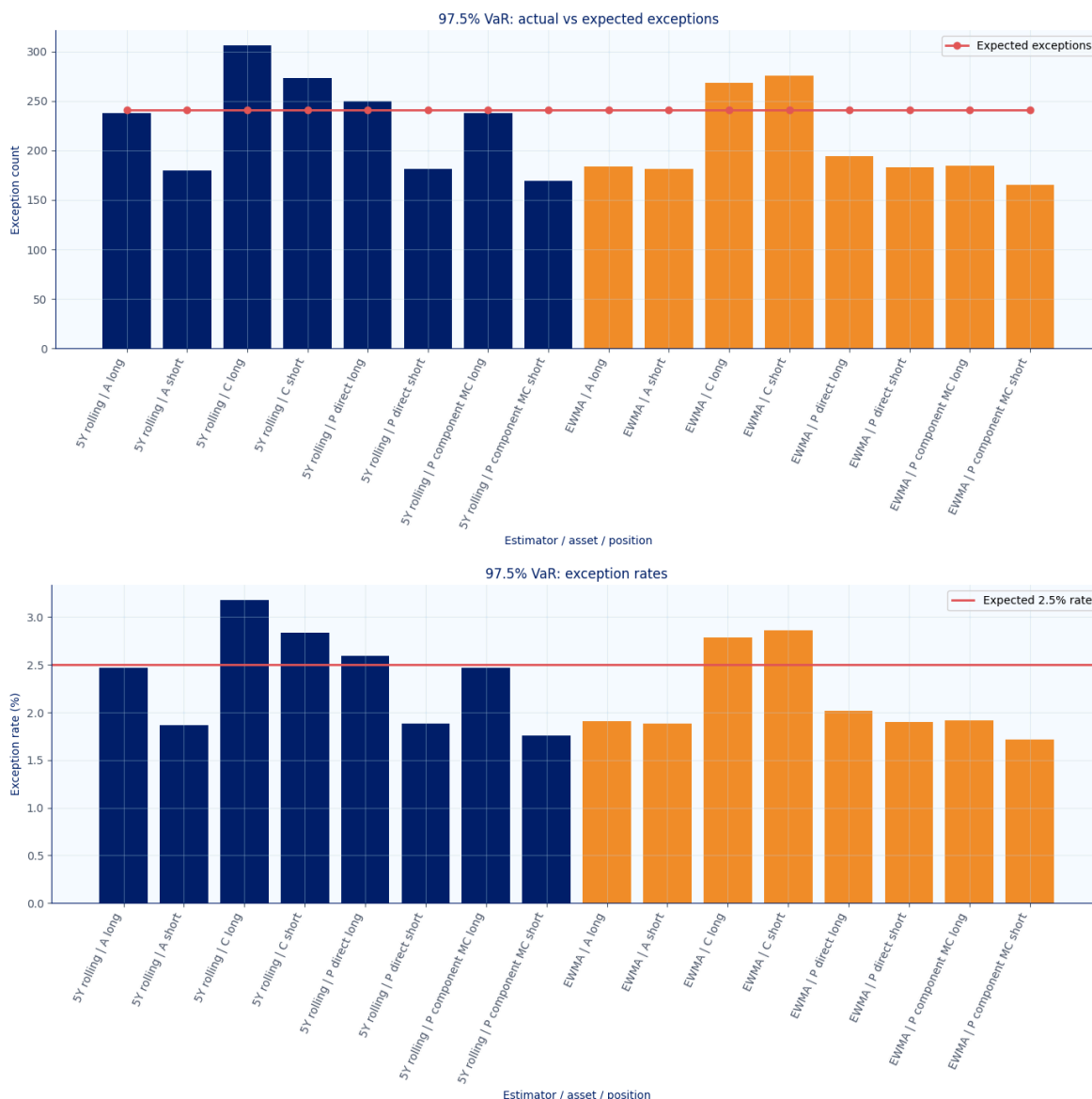
	VaR level	Estimator	Asset / method	Position	Backtest dates	Expected exceptions	Actual exception
0	0.990000	5Y rolling	A	long	9633	96.330000	15
1	0.990000	5Y rolling	A	short	9633	96.330000	9
2	0.990000	5Y rolling	C	long	9633	96.330000	18
3	0.990000	5Y rolling	C	short	9633	96.330000	13
4	0.990000	5Y rolling	P direct	long	9633	96.330000	16
5	0.990000	5Y rolling	P direct	short	9633	96.330000	9
6	0.990000	5Y rolling	P component MC	long	9633	96.330000	14
7	0.990000	5Y rolling	P component MC	short	9633	96.330000	8
8	0.990000	EWMA lambda=0.9989	A	long	9633	96.330000	11
9	0.990000	EWMA lambda=0.9989	A	short	9633	96.330000	8
10	0.990000	EWMA lambda=0.9989	C	long	9633	96.330000	13
11	0.990000	EWMA lambda=0.9989	C	short	9633	96.330000	13
12	0.990000	EWMA lambda=0.9989	P direct	long	9633	96.330000	12
13	0.990000	EWMA lambda=0.9989	P direct	short	9633	96.330000	7
14	0.990000	EWMA lambda=0.9989	P component MC	long	9633	96.330000	10
15	0.990000	EWMA lambda=0.9989	P component MC	short	9633	96.330000	7
16	0.975000	5Y rolling	A	long	9633	240.830000	23
17	0.975000	5Y rolling	A	short	9633	240.830000	18
18	0.975000	5Y rolling	C	long	9633	240.830000	30
19	0.975000	5Y rolling	C	short	9633	240.830000	27
20	0.975000	5Y rolling	P direct	long	9633	240.830000	25
21	0.975000	5Y rolling	P direct	short	9633	240.830000	18
22	0.975000	5Y rolling	P component MC	long	9633	240.830000	23
23	0.975000	5Y rolling	P component	short	9633	240.830000	16

	VaR level	Estimator	Asset / method	Position	Backtest dates	Expected exceptions	Actual exception
			MC				
24	0.975000	EWMA lambda=0.9989	A	long	9633	240.830000	18
25	0.975000	EWMA lambda=0.9989	A	short	9633	240.830000	18
26	0.975000	EWMA lambda=0.9989	C	long	9633	240.830000	26
27	0.975000	EWMA lambda=0.9989	C	short	9633	240.830000	27
28	0.975000	EWMA lambda=0.9989	P direct	long	9633	240.830000	19
29	0.975000	EWMA lambda=0.9989	P direct	short	9633	240.830000	18
30	0.975000	EWMA lambda=0.9989	P component MC	long	9633	240.830000	18
31	0.975000	EWMA lambda=0.9989	P component MC	short	9633	240.830000	16

```
In [30]: labels_975 = summary_975['Estimator'].str.replace(f' lambda={cfg.ewma_lambda}', '')
x_975 = np.arange(len(summary_975))

fig, ax = plt.subplots(figsize=(14, 7))
ax.bar(x_975, summary_975['Actual exceptions'], color=[COLUMBIA_NAVY if e
ax.plot(x_975, summary_975['Expected exceptions'], color=COLUMBIA_RED, ma
ax.set_title('97.5% VaR: actual vs expected exceptions')
ax.set_xlabel('Estimator / asset / position')
ax.set_ylabel('Exception count')
ax.set_xticks(x_975)
ax.set_xticklabels(labels_975, rotation=65, ha='right')
ax.legend()
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(14, 7))
ax.bar(x_975, 100 * summary_975['Exception rate'], color=[COLUMBIA_NAVY i
ax.axhline(100 * (1 - VAR_LEVEL_975), color=COLUMBIA_RED, linewidth=2.0,
ax.set_title('97.5% VaR: exception rates')
ax.set_xlabel('Estimator / asset / position')
ax.set_ylabel('Exception rate (%)')
ax.set_xticks(x_975)
ax.set_xticklabels(labels_975, rotation=65, ha='right')
ax.legend()
plt.tight_layout()
plt.show()
```

```
In [31]: fig, ax = plt.subplots(figsize=(13, 6))
ax.plot(df['Date'], df['VaR_long_P'], color=COLUMBIA_NAVY, linewidth=2.0,
ax.plot(df['Date'], df['VaR975_long_P'], color=COLUMBIA_NAVY, alpha=0.45,
ax.plot(df['Date'], df['ewma_VaR_long_P'], color=COLUMBIA_ORANGE, linewidth=2.0,
ax.plot(df['Date'], df['ewma_VaR975_long_P'], color=COLUMBIA_ORANGE, alpha=0.45,
ax.set_title('P direct long VaR: 99% vs 97.5%')
ax.set_xlabel('Date')
ax.set_ylabel('USD')
ax.legend()
ax.xaxis.set_major_locator(mdates.YearLocator(5))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.show()

fig, ax = plt.subplots(figsize=(13, 6))
ax.plot(df['Date'], df['VaR_short_P'], color=COLUMBIA_NAVY, linewidth=2.0,
ax.plot(df['Date'], df['VaR975_short_P'], color=COLUMBIA_NAVY, alpha=0.45,
ax.plot(df['Date'], df['ewma_VaR_short_P'], color=COLUMBIA_ORANGE, linewidth=2.0,
ax.plot(df['Date'], df['ewma_VaR975_short_P'], color=COLUMBIA_ORANGE, alpha=0.45,
ax.set_title('P direct short VaR: 99% vs 97.5%')
ax.set_xlabel('Date')
ax.set_ylabel('USD')
ax.legend()
ax.xaxis.set_major_locator(mdates.YearLocator(5))
```

```
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))  
plt.show()
```

